KU LEUVEN

FACULTEIT INDUSTRIËLE INGENIEURSWETENSCHAPPEN

KU LEUVEN – GENT

Simulatie van magnetische velden

Studenten: Arne Van der Stede & Syme Vandenbosch

Academiejaar 2025–2026



Inhoudsopgave

1 Introductie **2**

2 Theoretische achtergrond **3**
2.1 De wet van Biot–Savart . . . 3
2.2 Biot-Savart voor een eindige draad . . . 3
2.3 Uitwerking van Biot-Savart in Python . . . 5
2.4 Uitwerking van Biot-Savart in Cuda . . . 5

3 Magnetischeveldlijnen in 2D weergave **6**
3.1 Rechte draad simulatie . . . 6
3.1.1 Theoretisch . . . 6
3.1.2 Simulaties . . . 6
3.2 Lus simulatie . . . 7
3.2.1 Theoretisch . . . 7
3.2.2 Simulaties . . . 8
3.3 Solenoide simulatie . . . 9
3.3.1 Theoretisch . . . 9
3.3.2 Simulaties . . . 9
3.4 Besluiten bij magnetischeveldlijnen bij 2D-weergaves . . . 10
3.5 Code voor magnetischeveldlijnen in 2D-weergave . . . 10

4 Magnetischeveldlijnen in 3D-weergave **11**
4.1 Rechte draad simulatie . . . 11
4.2 Besluit bij rechte draad simulatie . . . 11
4.3 Lus simulatie . . . 11
4.4 Besluit bij lus simulatie . . . 12
4.5 Solenoid simulatie . . . 12
4.6 Besluit bij solenoïde simulatie . . . 12

5 Uitbreidingen **13**
5.1 Besluit bij uitbreidingen . . . 13
5.2 Vergelijking van de drie draadtopologieën en uitbreidingen over 250 000 gridpunten . . . 14
5.3 Besluit . . . 15

Bijlagen **16**

A Code voor 2D simulaties **16**

B Code voor de 3D-weergave **22**

1 Introductie

Voor het project hebben we gekozen voor het berekenen en visualiseren van een magnetisch veld dat ontstaat rond een enkele geleider, met de mogelijkheid om het te kunnen uitbreiden naar complexere opstellingen zoals een stroomlus of zelfs spoelen.

De berekeningen van het magnetische veld is gebaseerd op de wet van Biot-Savart. Deze wet stelt dat het magnetisch veld in een punt in de ruimte wordt bepaald door de som van de bijdragen van alle stroomelementen in de geleider. Het bepalen hiervan resulteert in een groot aantal berekeningen. Hoe meer punten er genomen worden over de geleider hoe meer berekeningen, maar ook wordt de visualisatie gedetailleerder. Dit resulteert aan de ene kant in een hogere rekentijd (waar de GPU dus tijdswinst zou kunnen boeken t.o.v. de CPU) en anderzijds ook in een meer gedetailleerde en nauwkeurige voorstelling van het magnetisch veld. Voor elk punt in de ruimte moeten immers vectoriële berekeningen worden uitgevoerd, met meerdere integratiestappen en sommaties.

Hierdoor is het een geschikt project voor parallelle verwerking op een GPU. Elk punt in de ruimte kan onafhankelijk van de andere punten berekend worden aangezien er geen afhankelijkheid is tussen de punten onderling. Hierdoor kunnen de berekeningen opgesplitst kunnen worden in duizenden of zelfs miljoenen gelijktijdige taken.

Door gebruik te maken van CUDA kunnen we deze parallelle architectuur benutten om de berekeningen van het magnetisch veld aanzienlijk te versnellen. Dit is vooral voordelig wanneer gewerkt wordt met hoge resoluties of complexere geometrieën van draad vormen, waarbij de rekentijd op een CPU snel kan oplopen. Naast de performantieverbetering maakt GPU-versnelling het ook mogelijk om interactievere en visueel rijkere simulaties te realiseren. De berekende veldwaarden kunnen efficiënt worden doorgestuurd naar een Python-gebaseerde visualisatie, waardoor inzicht verkregen wordt in de structuur en sterkte van het magnetisch veld.

Dit verslag beschrijft de implementatie van de magnetische veldsimulaties, de analyse van de prestatie waarbij de CPU gebruikt wordt t.o.v. de GPU implementatie en wordt de resultaten gevisualiseerd. Daarbij ligt de nadruk op de computationele efficiëntie van de oplossing. De broncode van dit project is beschikbaar via de GitHub-repository [1].

Url naar GitHub-repository: https://github.com/ArneVanderStede/MagneticFieldSimulator

2 Theoretische achtergrond

Voordat er over kan gegaan worden tot het ontwikkelen van de code is het noodzakelijk dat er voldoende theoretische kennis over het onderwerp is opgedaan.

2.1 De wet van Biot–Savart

De wet van Biot–Savart beschrijft hoe een elektrisch stroomvoerende geleider een magnetisch veld opwekt in de ruimte. Deze wet legt de fundamenten in elektromagnetisme en maakt het mogelijk om een magnetisch veld te berekenen op basis van de gekozen vorm van geleider en de stroomgrootte die erdoor vloeit.
De wet stelt dat elk infinitesimaal klein stuk van de gekozen stroomvoerdende geleider, een bijdrage aan het magnetisch veld heeft op een welbepaald punt in de ruimte. Waarbij de grootte van de stroom, de lengte en oriëntatie bepaald hoeveel de bijdrage wordt. Wiskundig wordt dit uitgedrukt als [2]:

$$d\vec{B} = \frac{\mu_0}{4\pi} \cdot \frac{I \cdot d\vec{l} \times \hat{r}}{r^2} \tag{1}$$

Hierbij is:

- $d\vec{B}$: de infinitesimale bijdrage tot het magnetisch veld [T]
- μ_0: de magnetische permeabiliteit van het vacuüm [$\mathrm{H\,m^{-1}}$]
- I: de elektrische stroom door de geleider [A]
- $d\vec{l}$: een infinitesimaal stroomelement, met richting van de stroom [m]
- $\hat{r}$: de eenheidsvector van het stroomelement naar het beschouwde punt [-]
- r: de afstand tussen het stroomelement en het beschouwde punt [m]

Hierbij stelt $d\vec{B}$ de infinitesimale bijdrage tot het magnetisch veld voor, μ_0 de magnetische permeabiliteit van het vacuüm, $d\vec{l}$ het stroomelement, $\hat{r}$ de eenheidsvector van het stroomelement naar het beschouwde punt, en r de afstand tussen beide.
Om het totale magnetisch veld te bepalen, moeten alle bijdragen van de verschillende stroomelementen langs de volledige geleider opgeteld worden. Dit gebeurt via een lijnintegraal over de geleider:

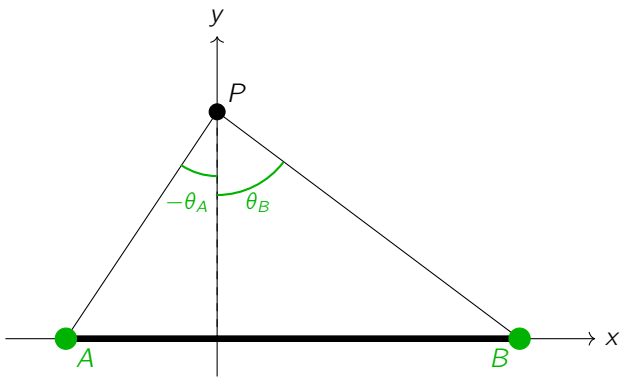
$$\vec{B} = \int_C d\vec{B} = \frac{\mu_0}{4\pi} \int_C I \frac{d\vec{l} \times \vec{r}}{r^3} \tag{2}$$

Vaak wordt de integraal niet analytisch oplosbaar. Daarom wordt er gebruik gemaakt van numerieke benadering, waarbij de geleider opgedeeld wordt in een groot aantal kleine segmenten. Vervolgens worden de bijdrages van de deeltjes bij elkaar opgeteld en dit bepaald dan het magnetisch veld. Intuïtief kunnen de veldlijnen bepaald worden met de rechterhandregel.

2.2 Biot-Savart voor een eindige draad

Wanneer de integraal niet analytisch oplosbaar is wordt er gebruik gemaakt door de wet van Biot-Savart toe te passen op een draad met eindige lengte. Hieronder staat de afleiding voor het bekomen van de wet van Biot-Savart over een eindige draad beschreven:

Vertrekpunt: Wet van Biot-Savart over een eindige draad



Figuur 1: Geometrie voor de Biot-Savart wet bij een eindig rechtlijnig stroomsegment van A naar B.

De wet van Biot-Savart voor een eindig stroomsegment dat van punt A naar punt B loop luidt [2] en is wordt afgebeeld in Figuur 1:

$$\vec{B}_{segment} = \frac{\mu_0 I}{4\pi} \frac{\vec{r}_A \times \vec{r}_B}{|\vec{r}_A \times \vec{r}_B|^2} (\cos\theta_A - \cos\theta_B) \tag{3}$$

waarbij:

- $\vec{r}_A = \vec{P} - \vec{A}$ de vector van het startpunt $\vec{A}$ van het segment naar het veldpunt $\vec{P}$,
- $\vec{r}_B = \vec{P} - \vec{B}$ de vector van het eindpunt $\vec{B}$ van het segment naar $\vec{P}$,
- θ_A en θ_B de hoeken die respectievelijk $\vec{r}_A$ en $\vec{r}_B$ maken met de segmentrichting $\hat{l}$.

Stap 1: Cosinussen via dot-producten

De eenheidsvector langs het segment is:

$$\hat{l} = \frac{\vec{B} - \vec{A}}{|\vec{B} - \vec{A}|} \tag{4}$$

Via de definitie van het dot-product volgt dan:

$$\cos\theta_A = \frac{\vec{r}_A \cdot \hat{l}}{|\vec{r}_A|} = \frac{\vec{r}_A \cdot (\vec{B} - \vec{A})}{|\vec{r}_A|\,|\vec{B} - \vec{A}|} \tag{5}$$

$$\cos\theta_B = \frac{\vec{r}_B \cdot \hat{l}}{|\vec{r}_B|} = \frac{\vec{r}_B \cdot (\vec{B} - \vec{A})}{|\vec{r}_B|\,|\vec{B} - \vec{A}|} \tag{6}$$

Het verschil wordt dan:

$$\cos\theta_A - \cos\theta_B = \frac{\left(\frac{\vec{r}_A \cdot (\vec{B} - \vec{A})}{|\vec{r}_A|} - \frac{\vec{r}_B \cdot (\vec{B} - \vec{A})}{|\vec{r}_B|}\right)}{|\vec{B} - \vec{A}|}. \tag{7}$$

Stap 2: Vereenvoudiging uitbrengen $\vec{B} - \vec{A} = \vec{r}_A - \vec{r}_B$

Merk op dat:

$$\vec{r}_A - \vec{r}_B = (\vec{P} - \vec{A}) - (\vec{P} - \vec{B}) = \vec{B} - \vec{A} \tag{8}$$

Substitueer Vergelijking 8 in de dot-producten:

$$\vec{r}_A \cdot (\vec{r}_A - \vec{r}_B) = |\vec{r}_A|^2 - \vec{r}_A \cdot \vec{r}_B \tag{9}$$

$$\vec{r}_B \cdot (\vec{r}_A - \vec{r}_B) = \vec{r}_A \cdot \vec{r}_B - |\vec{r}_B|^2 \tag{10}$$

De teller uit Vergelijking 7 wordt dan:

$$\frac{|\vec{r}_A|^2 - \vec{r}_A \cdot \vec{r}_B}{|\vec{r}_A|} - \frac{\vec{r}_A \cdot \vec{r}_B - |\vec{r}_B|^2}{|\vec{r}_B|} \tag{11}$$

Wanneer we deze teller op een gemeenschappelijke noemer zetten verkrijgen we:

$$= \frac{\left(|\vec{r}_A|^2 - \vec{r}_A \cdot \vec{r}_B\right)|\vec{r}_B| - \left(\vec{r}_A \cdot \vec{r}_B - |\vec{r}_B|^2\right)|\vec{r}_A|}{|\vec{r}_A|\,|\vec{r}_B|} \tag{12}$$

Het uitwerken van deze teller levert:

$$\begin{aligned} &= \frac{|\vec{r}_A|^2|\vec{r}_B| - (\vec{r}_A \cdot \vec{r}_B)|\vec{r}_B| - (\vec{r}_A \cdot \vec{r}_B)|\vec{r}_A| + |\vec{r}_B|^2|\vec{r}_A|}{|\vec{r}_A|\,|\vec{r}_B|} \\ &= \frac{|\vec{r}_A||\vec{r}_B|\left(|\vec{r}_A| + |\vec{r}_B|\right) - (\vec{r}_A \cdot \vec{r}_B)\left(|\vec{r}_A| + |\vec{r}_B|\right)}{|\vec{r}_A|\,|\vec{r}_B|} \\ &= \frac{\left(|\vec{r}_A| + |\vec{r}_B|\right)\left(|\vec{r}_A||\vec{r}_B| - \vec{r}_A \cdot \vec{r}_B\right)}{|\vec{r}_A|\,|\vec{r}_B|} \end{aligned} \tag{13}$$

Waaruit volgt dat:

$$\cos\theta_A - \cos\theta_B = \frac{1}{|\vec{B} - \vec{A}|} \cdot \frac{\left(|\vec{r}_A| + |\vec{r}_B|\right)\left(|\vec{r}_A||\vec{r}_B| - \vec{r}_A \cdot \vec{r}_B\right)}{|\vec{r}_A|\,|\vec{r}_B|} \tag{14}$$

Stap 3: Vereenvoudiging van $|\vec{r}_A \times \vec{r}_B|^2$ uit de noemer van Vergelijking 3

Gebruik de vectoridentiteit [3]:

$$|\vec{r}_A \times \vec{r}_B|^2 = |\vec{r}_A|^2|\vec{r}_B|^2 - (\vec{r}_A \cdot \vec{r}_B)^2 = \big(|\vec{r}_A||\vec{r}_B| - \vec{r}_A \cdot \vec{r}_B\big)\big(|\vec{r}_A||\vec{r}_B| + \vec{r}_A \cdot \vec{r}_B\big) \tag{15}$$

Stap 4: Alles invullen en vereenvoudigen

Substitueer Vergelijkingen (11) en (12) in Vergelijking (1):

$$\vec{B}_{segment} = \frac{\mu_0 I}{4\pi} \cdot \frac{\vec{r}_A \times \vec{r}_B}{\big(|\vec{r}_A||\vec{r}_B| - \vec{r}_A \cdot \vec{r}_B\big)\big(|\vec{r}_A||\vec{r}_B| + \vec{r}_A \cdot \vec{r}_B\big)} \cdot \frac{\big(|\vec{r}_A| + |\vec{r}_B|\big)\big(|\vec{r}_A||\vec{r}_B| - \vec{r}_A \cdot \vec{r}_B\big)}{|\vec{r}_A|\,|\vec{r}_B|\,|\vec{B} - \vec{A}|} \tag{16}$$

De factor $\big(|\vec{r}_A||\vec{r}_B| - \vec{r}_A \cdot \vec{r}_B\big)$ valt weg in teller en noemer. Verder geldt $|\vec{B} - \vec{A}| = |\vec{r}_A - \vec{r}_B|$, wat eveneens wegvalt. Dit geeft de eindvorm:

Eindresultaat

$$\boxed{\vec{B}_{segment} = \frac{\mu_0 I}{4\pi} \cdot (\vec{r}_A \times \vec{r}_B) \cdot \frac{|\vec{r}_A| + |\vec{r}_B|}{|\vec{r}_A|\,|\vec{r}_B|\big(|\vec{r}_A|\,|\vec{r}_B| + \vec{r}_A \cdot \vec{r}_B\big)}} \tag{17}$$

2.3 Uitwerking van Biot-Savart in Python

De analytische eindvorm uit Vergelijking 17 wordt numeriek geïmplementeerd door de geleider op te delen in N kleine rechtlijnige eindige segmenten. Voor elk segment k met startpunt $\vec{A}_k$ en eindpunt $\vec{B}_k$ worden de vectoren $\vec{r}_{A_k} = \vec{P} - \vec{A}_k$ en $\vec{r}_{B_k} = \vec{P} - \vec{B}_k$ berekend voor elk veldpunt $\vec{P}$ in het rekenrooster. De totale bijdrage wordt dan gesommeerd over alle segmenten k:

$$\vec{B}(P) = \frac{\mu_0 I}{4\pi} \sum_k \frac{(\vec{r}_{A_k} \times \vec{r}_{B_k}) \cdot (|\vec{r}_{A_k}| + |\vec{r}_{B_k}|)}{|\vec{r}_{A_k}|\,|\vec{r}_{B_k}|\,(|\vec{r}_{A_k}|\,|\vec{r}_{B_k}| + \vec{r}_{A_k} \cdot \vec{r}_{B_k})} \tag{18}$$

2.4 Uitwerking van Biot-Savart in Cuda

Naast de CPU-implementatie in NumPy is een GPU-versnelde versie beschikbaar via een CUDA-kernel in CuPy. De kernel berekent de bijdrage van elk segment per veldpunt parallel op de GPU, waarbij elk CUDA-thread verantwoordelijk is voor één veldpunt. Ook hier wordt dezelfde vergelijking als bij de Python uitwerking gebruikt. De veldlijnen worden bereken door de draad op te delen in verschillende eindige draad stukken en daar de som van te nemen.

$$\vec{B}(P) = \frac{\mu_0 I}{4\pi} \sum_k \frac{(\vec{r}_{A_k} \times \vec{r}_{B_k}) \cdot (|\vec{r}_{A_k}| + |\vec{r}_{B_k}|)}{|\vec{r}_{A_k}|\,|\vec{r}_{B_k}|\,(|\vec{r}_{A_k}|\,|\vec{r}_{B_k}| + \vec{r}_{A_k} \cdot \vec{r}_{B_k})} \tag{19}$$

Dit is exact Vergelijking 17 toegepast op één segment k.

3 Magnetischeveldlijnen in 2D weergave

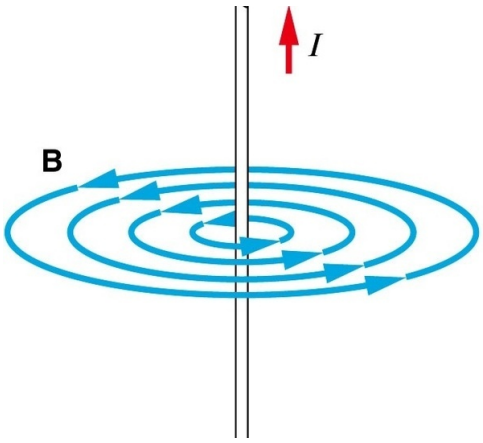
Voordat er een complexe 3D simulatie uitgevoerd wordt, is er gestart met het weergeven van de veldlijnen rond een enkele geleider. Voor de berekeningen wordt eerst gebruik gemaakt van de CPU. Later zal dit uitgevoerd worden op de GPU, om zo te kijken wat de tijdwinst is. Visualisaties zijn gemaakt via Python.

3.1 Rechte draad simulatie

Als eerste onderdeel worden de veldlijnen voor een rechte draad bepaald.

3.1.1 Theoretisch

Voor de simulatie van de geleider wordt er een veldlijnen verloop verwacht zoals te zien is in Figuur: 2
In de simulatie zien we een gelijkaardig patroon. De veldlijnen worden met behulp van pijlen weergegeven. De grootte van de pijl geeft aan hoe sterk het veld is op elk punt.

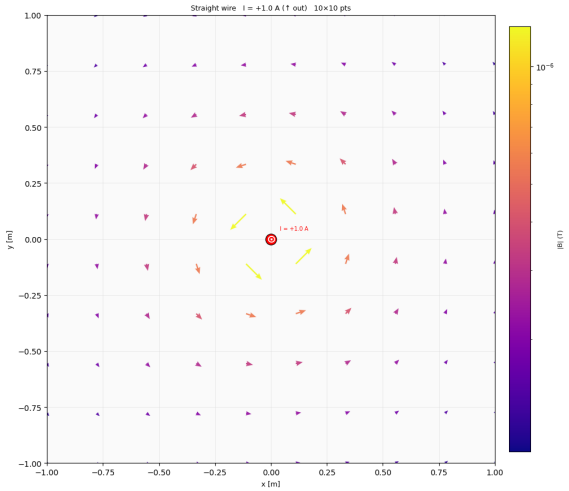


Figuur 2: Magnetische veldlijnen in een rechte draad

3.1.2 Simulaties

Zoals te zien is in Figuur 3 loopt de stroom in de figuur en lopen de veldlijnen gelijk met de veldlijnen die in Figuur 2 te zien zijn.

- Visueel:



Figuur 3: Magnetische veldlijnen in een rechte draad

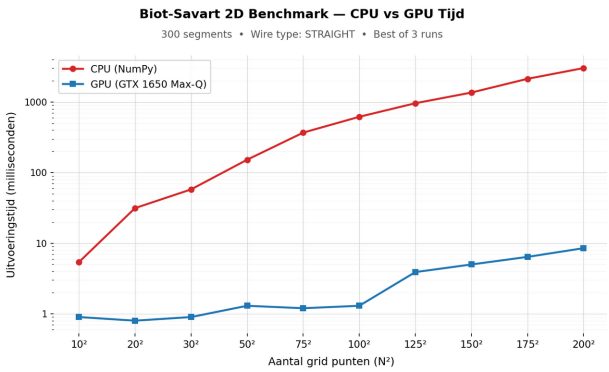
- Benchmarks:

  Als volgende stap voegen we ook GPU berekeningen toe. We geven het verschil voor verschillende hoeveelheden van punten in een grafiek weer.

```
============================================================
  Wire type : STRAIGHT   S=300 segments
  CPU       : NumPy
  GPU       : NVIDIA GeForce GTX 1650 with Max-Q Design
  Runs      : best of 3
============================================================
  Grid    Points          Ops    CPU (ms)    GPU (ms)   Speedup
------------------------------------------------------------
  10²        100       30,000         5.4         0.9      6.0x
  20²        400      120,000        31.4         0.8     38.8x
  30²        900      270,000        57.8         0.9     65.3x
  50²      2,500      750,000       152.2         1.3    114.3x
  75²      5,625    1,687,500       368.3         1.2    302.2x
 100²     10,000    3,000,000       617.0         1.3    489.1x
 125²     15,625    4,687,500       960.3         3.9    246.2x
 150²     22,500    6,750,000      1360.2         5.0    273.8x
 175²     30,625    9,187,500      2128.7         6.4    333.9x
 200²     40,000   12,000,000      3021.9         8.5    355.6x
============================================================
```

Figuur 4: Reslutaten van de rechte draad

- Grafisch:



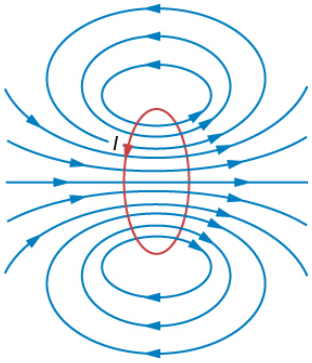
Figuur 5: Resultaten grafisch weergegeven voor de simulatie van een rechte draad

3.2 Lus simulatie

Als tweede test proberen we eens een lus te simuleren.

3.2.1 Theoretisch

Theoretisch verwachten we dat de veldlijnen er uitzien zoals bij Figuur:6.

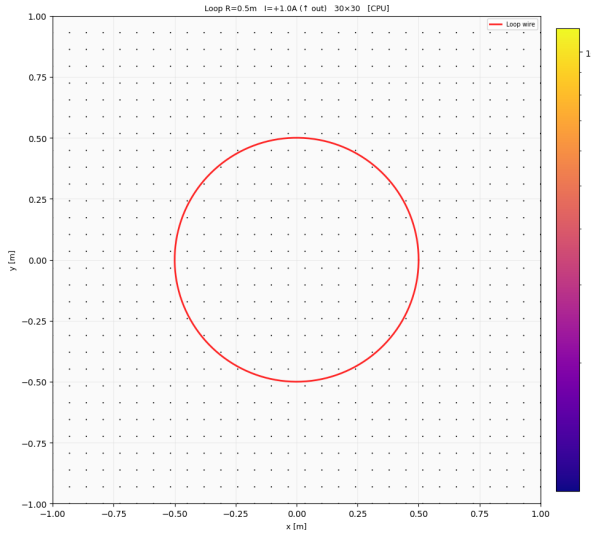


Figuur 6: Magnetische veldlijnen in een lus

3.2.2 Simulaties

In de simulatie zien we een gelijkaardig patroon. De veldlijnen worden met behulp van pijlen weergegeven. De grootte van de pijl geeft aan hoe sterk het veld is op elk punt. Het is duidelijk dat de stroom doorheen de lus loop en hierdoor zijn de pijlen steeds loodrecht invallend op het blad of uitkomend. Dit is te zien in Figuur 7.

- Visueel:



Figuur 7: Magnetische veldlijnen voor een lus bij simulatie

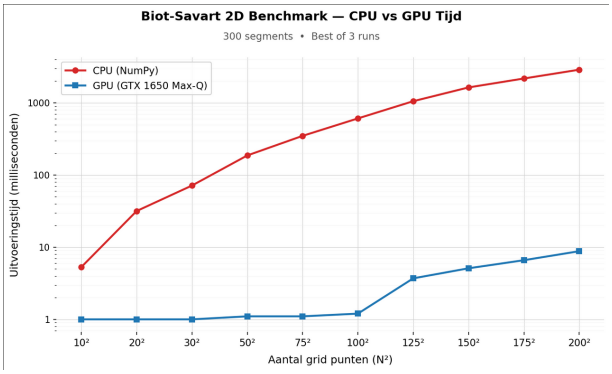
- Benchmarks:

  Als volgende stap voegen we ook GPU berekeningen toe. We geven het verschil voor verschillende hoeveelheden van punten in een grafiek weer.

```
================================================================
  Biot-Savart 2D Benchmark  —  S=300 segments
  CPU : NumPy
  GPU : NVIDIA GeForce GTX 1650 with Max-Q Design
  Runs: best of 3
================================================================
  Grid    Points          Ops    CPU (ms)    GPU (ms)   Speedup
----------------------------------------------------------------
  [warming up GPU kernel...]
  10²        100       30,000         4.8         0.9      5.5x
  20²        400      120,000        32.0         0.6     52.4x
  30²        900      270,000        59.9         0.6    103.2x
  50²      2,500      750,000       162.8         0.8    197.6x
  75²      5,625    1,687,500       353.2         0.7    518.1x
 100²     10,000    3,000,000       703.5         0.9    810.5x
 125²     15,625    4,687,500      1072.9         2.4    441.2x
 150²     22,500    6,750,000      1690.4         3.4    495.2x
 175²     30,625    9,187,500      2206.9         4.4    501.4x
 200²     40,000   12,000,000      2708.4         5.3    512.3x
================================================================
```

Figuur 8: Reslutaten van de lus

- Grafisch:

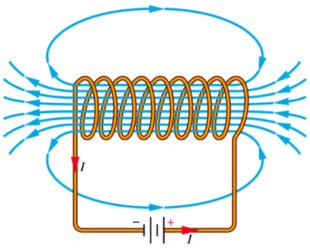


Figuur 9: Resultaten grafisch weergegeven voor de simulatie van een lus

3.3 Solenoide simulatie

3.3.1 Theoretisch

Als laatste 2D test is er een solenoïde gesimuleerd. Hierbij verwachten we dat de veldlijnen er uitzien zoals bij Figuur 10.

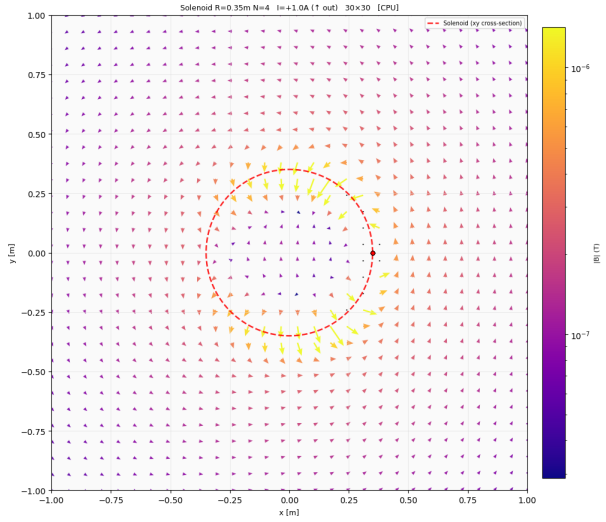


Figuur 10: Magnetische veldlijnen bij een solenoïde

3.3.2 Simulaties

Zoals te zien is in Figuur 11 loopt de stroom in de figuur en lopen de veldlijnen gelijk met de veldlijnen die in Figuur 10 te zien zijn.

- Visueel:



Figuur 11: Magnetische veldlijnen van een solenoide bij simulatie

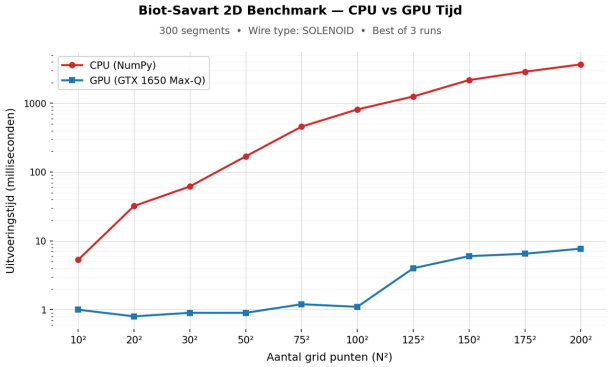
- Benchmarks:

  Als volgende stap voegen we ook GPU berekeningen toe. We geven het verschil voor verschillende hoeveelheden van punten in een grafiek weer.

```
============================================================
  Wire type : SOLENOID   S=300 segments
  CPU       : NumPy
  GPU       : NVIDIA GeForce GTX 1650 with Max-Q Design
  Runs      : best of 3
============================================================
  Grid    Points          Ops    CPU (ms)    GPU (ms)   Speedup
------------------------------------------------------------
  10²        100       30,000         5.3         1.0      5.2x
  20²        400      120,000        32.1         0.8     38.3x
  30²        900      270,000        61.8         0.9     65.5x
  50²      2,500      750,000       169.1         0.9    191.2x
  75²      5,625    1,687,500       457.5         1.2    395.1x
 100²     10,000    3,000,000       813.6         1.1    717.0x
 125²     15,625    4,687,500      1253.8         4.0    312.0x
 150²     22,500    6,750,000      2175.2         6.0    359.8x
 175²     30,625    9,187,500      2872.3         6.5    441.1x
 200²     40,000   12,000,000      3672.5         7.7    478.9x
============================================================
```

Figuur 12: Reslutaten van de rechte draad

- Grafisch:



Figuur 13: Resultaten grafisch weergegeven voor de simulatie van een lus

3.4 Besluiten bij magnetischeveldlijnen bij 2D-weergaves

Uit de resultaten uit de benchmarks bij de vergelijking van de drie draadtypes (rechte draad, lus en solenoïde) blijkt dat de GPU-implementatie aanzienlijke verbeteringen geeft ten opzichte van de CPU-implementatie. Voor kleinere *grids* behaald de GPU al een opmerkelijke *speedup* ten opzichte van de CPU. Maar bij grotere *grids* is de tijdswinst pas echt te merken.

De CPU-uitvoeringstijd schaalt ongeveer lineair met het aantal operaties. De GPU schaalt logaritmisch met het aantal operaties. Dit toont aan dat GPU-acceleratie voor Biot-Savart-berekeningen zeer efficiënt kan uitgevoerd worden door gebruikt te maken van een GPU.

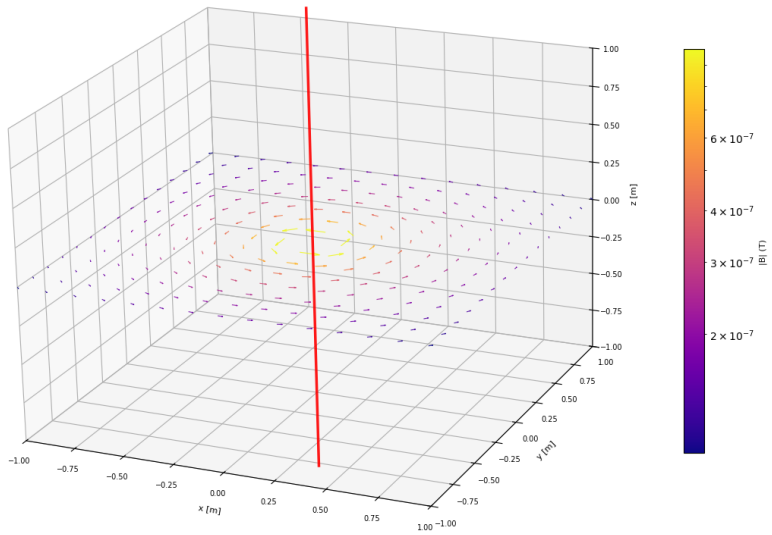
3.5 Code voor magnetischeveldlijnen in 2D-weergave

In de 2D-simulatie kunnen we niet alle veldlijnen controleren aangezien er telkens maar vanuit één perspectief gekeken kan worden. Als volgende stap schakelen we voor de weergave over naar een 3D-voorstelling. De code voor het plotten en benchmarken van de geleider, lus en solenoïde zijn weergegeven in Bijlage A.

4 Magnetischeveldlijnen in 3D-weergave

Door gebruik te maken van een interactieve 3D-weergave wordt de vorming van de veldlijnen duidelijker zichtbaar. Bij onderstaande figuren wordt telkens een weergave getoond van de veldlijnen voor één laag (een doorsnede doorheen de opstelling). Op deze manier lopen er niet te veel lijnen door elkaar en blijft het overzicht behouden. Een andere mogelijkheid is om alle veldlijnen weer te geven, deze methode is zwaarder voor de CPU. De code is weergegeven in Bijlage B.

4.1 Rechte draad simulatie

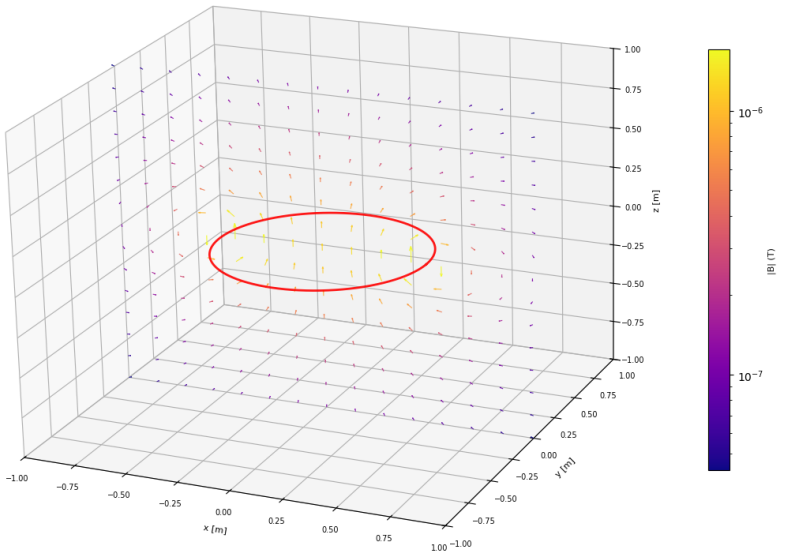


Figuur 14: 3D simulatie van een rechte draad

4.2 Besluit bij rechte draad simulatie

Uit de 3D-weergave blijkt duidelijk dat de veldlijnen in een cirkelvormige beweging rond de geleider vloeien. De richting wordt bepaald door de stroomrichting: in dit geval stroomt de stroom omhoog door de geleider. Via de rechterhandregel volgt dat de veldlijnen tegen de wijzers van de klok in gericht zijn, als de opstelling van bovenaf bekeken wordt.

4.3 Lus simulatie

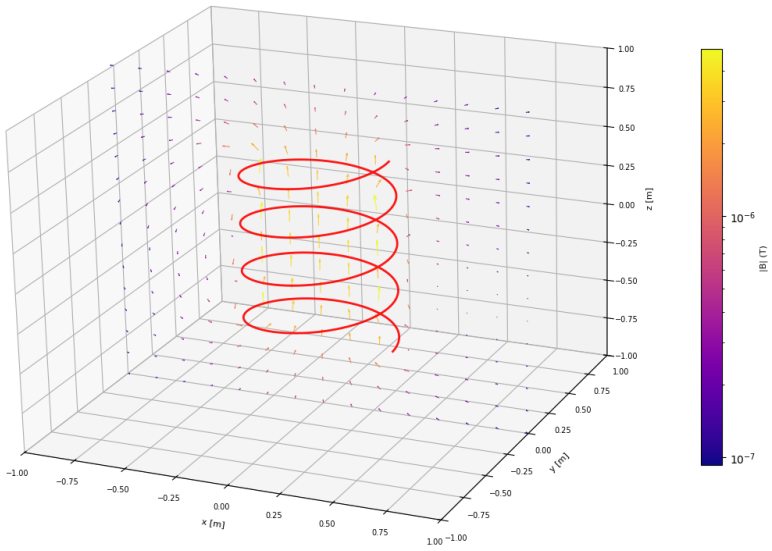


Figuur 15: 3D simulatie van een lus

4.4 Besluit bij lus simulatie

In de 3D-weergave is te zien dat de veldlijnen cirkels vormen die loodrecht op de lus staan. Ook hier kan de rechterhandregel worden toegepast om de richting te bepalen, al is dit minder intuïtief dan bij een rechte draad. De stroom in de lus vloeit tegen de wijzers van de klok in (van bovenaf bekeken), waardoor de veldlijn-cirkels van onder naar boven gericht zijn.

4.5 Solenoid simulatie



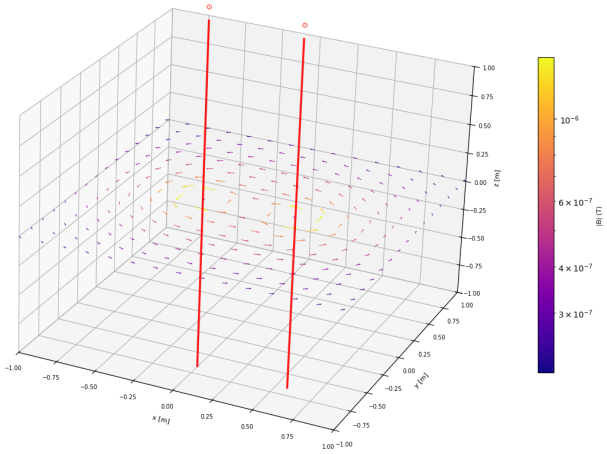
Figuur 16: 3D simulatie van een solenoïde

4.6 Besluit bij solenoïde simulatie

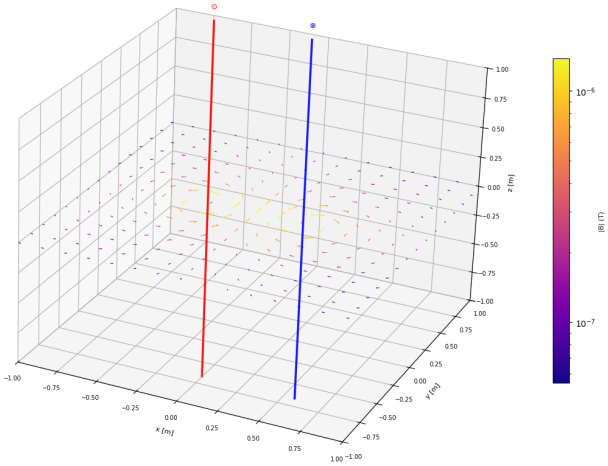
Ook in deze 3D-weergave wordt de stroomrichting van de veldlijnen duidelijk zichtbaar. Ze vloeien van onder naar boven. Dit kan opnieuw worden vastgesteld met de rechterhandregel. Doordat de veldlijnen van de afzonderlijke wikkelingen elkaar plaatselijk kunnen versterken of verzwakken, is het echter minder eenvoudig om de resulterende richting puur intuïtief te bepalen.

5 Uitbreidingen

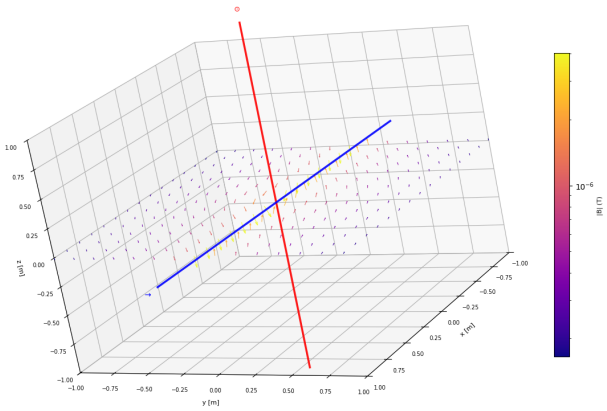
Voor de volgende stap voegen we een extra geleider toe om te kijken naar de invloed ervan. Theoretisch gezien zou het magnetische veld van de ene geleider moeten reageren op de andere.



Figuur 17: 3D simulatie van een 2 rechte draden waarbij de stroom in dezelfde richting loop



Figuur 18: 3D simulatie van een 2 rechte draden waarbij de stroom in tegengestelde richting loop

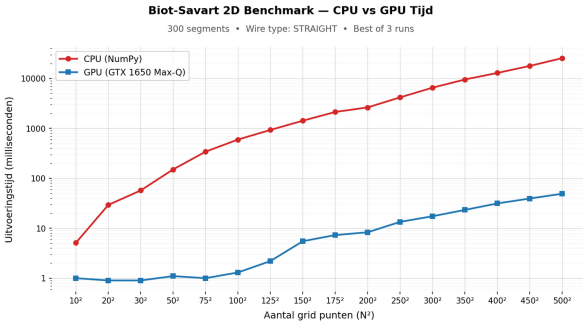


Figuur 19: 3D simulatie van een 2 rechte draden die 90° verdraaid is

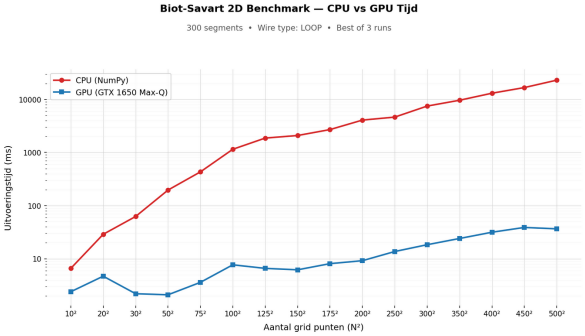
5.1 Besluit bij uitbreidingen

Er is telkens te zien dat zoals theoretisch ook verwacht wordt dat de veldlijnen met elkaar interageren en elkaar beïnvloeden.

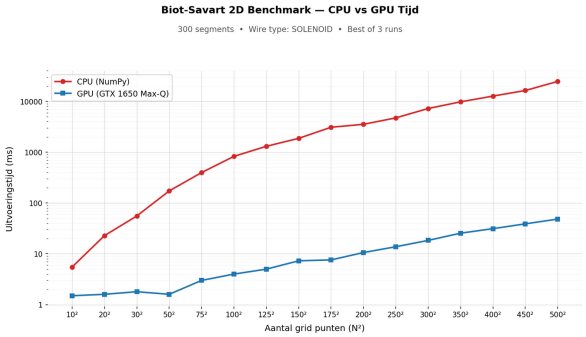
5.2 Vergelijking van de drie draadtopologieën en uitbreidingen over 250 000 gridpunten



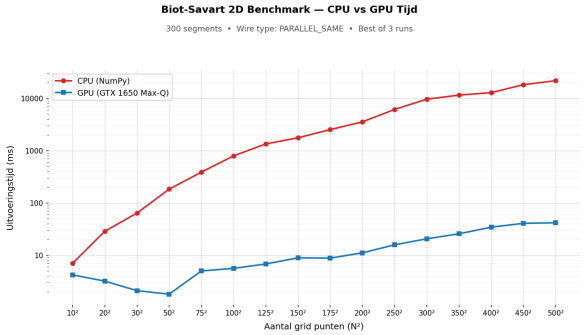
Enkelvoudige rechte geleider



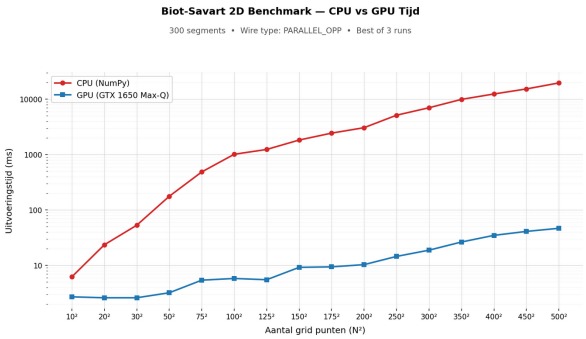
Stroomlus



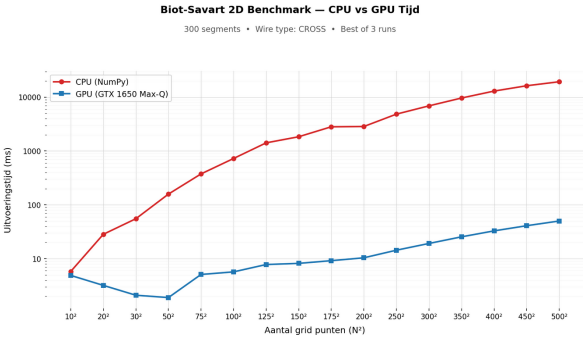
Solenoïde



Parallelle geleiders met stroom in dezelfde richting



Parallelle geleiders met stroom in tegengestelde richting



Twee geleiders in gekruiste opstelling

Figuur 20: Vergelijking van de CPU- en GPU-uitvoeringstijd voor zes draadtopologieën tot $N^2 = 250\,000$ gridpunten (300 segmenten, best of 3 runs).

5.3 Besluit

Uit een vergelijking van de zes geteste topologieën blijkt dat de GPU-implementatie consistent sneller is dan de CPU-implementatie op basis van NumPy. De GPU-uitvoeringstijd loopt lineair op van circa 1–5 ms bij kleine grids (10^2) tot 40–50 ms bij (500^2) gridpunten, terwijl de CPU-tijd min of meer logaritmische toeneemt tot 19–26 seconden bij hetzelfde gridformaat.

Dit verschil is te verklaren door de parallelle architectuur van de GPU: elk gridpunt wordt door een afzonderlijke CUDA-thread verwerkt, waardoor alle N^2 punten gelijktijdig worden berekend. De CPU voert de berekening daarentegen sequentieel uit, met een totale complexiteit van $\mathcal{O}(N^2 \cdot S)$. Met $N \cdot N = N^2$ het aantal elementen en S het aantal stappen die doorlopen dienen te worden

De maximale speedup varieert van 385× tot 628× bij 500^2 gridpunten. De eenvoudige topologieën zoals de rechte geleider (520×) en de solenoïde (510×) presteren beter doordat hun regelmatige geheugenstructuur de GPU-coalescentie bevordert. Complexere uitbreidingsconfiguraties zoals gekruist en parallel vereisen niet-aaneengesloten geheugentoegangen, wat de GPU-efficiëntie verlaagt. Bij alle topologieën is een knikpunt zichtbaar rond 125^2–150^2: onder dit punt past de volledige dataset in het snelle GPU-geheugen, erboven stijgt de uitvoeringstijd door toenemende host-device data uitwisseling.

Bij de 3D-weergave vertraagt de applicatie sterk naarmate we meer punten weergeven in de GUI. Dit komt doordat het plotten in matplotlib via de CPU verloopt en daardoor een bottleneck vormt bovenop de berekening van het magnetische veld. Vooral wanneer het aantal weer te geven vectoren stijgt, moet de applicatie niet alleen meer data verwerken, maar ook meer grafische objecten opbouwen en renderen, wat de interactie trager maakt.

Voor de interactieve toepassing is het daarom beter om het aantal getoonde vectoren te beperken en de volledige resolutie enkel te gebruiken wanneer absolute detailweergave nodig is. Een mogelijke uitbreiding van dit project is om de visualisatie volgens de GPU te laten lopen.

Bijlagen

A Code voor 2D simulaties

```
import numpy as np
import time

BENCHMARK = True

# benchmark settings
BENCH_GRIDS   = [10, 20, 30, 50, 75, 100, 125, 150, 175, 200]
BENCH_REPEATS = 3
BENCH_WIRES   = ["straight", "loop", "solenoid"]


MU0              = 4e-7 * np.pi
EXTENT           = 1.0
LOOP_RADIUS      = 0.5
SOLENOID_RADIUS  = 0.35
SOLENOID_LENGTH  = 1.2
SOLENOID_TURNS   = 4

# GPU detection
try:
    import cupy as cp
    cp.cuda.Device(0).use()
    _ = cp.array([1.0], dtype=cp.float32)
    HAS_GPU   = True
    gpu_props = cp.cuda.runtime.getDeviceProperties(0)
    GPU_NAME  = gpu_props["name"].decode() if isinstance(gpu_props["name"], bytes) else
        str(gpu_props["name"])
except Exception:
    HAS_GPU  = False
    cp       = None
    GPU_NAME = "Not available"

# CUDA kernel
_CUDA_SRC = r"""
extern "C" __global__
void biot_savart_2d(
    const float* __restrict__ pts,
    const float* __restrict__ Ax,
    const float* __restrict__ Ay,
    const float* __restrict__ Az,
    const float* __restrict__ Bx,
    const float* __restrict__ By,
    const float* __restrict__ Bz,
    float*       __restrict__ Bout,
    const int num_points, const int num_segments,
    const float prefactor, const float mask_radius)
{
    int point_idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (point_idx >= num_points) return;

    const float pt_x = pts[point_idx*2];
    const float pt_y = pts[point_idx*2+1];
    const float pt_z = 0.f;
    float field_x = 0.f, field_y = 0.f;
    float min_dist_sq = 1e30f;

    for (int k = 0; k < num_segments; ++k) {
        const float start_x = Ax[k], start_y = Ay[k], start_z = Az[k];
        const float end_x   = Bx[k], end_y   = By[k], end_z   = Bz[k];

        const float to_start_x = pt_x-start_x, to_start_y = pt_y-start_y, to_start_z =
            pt_z-start_z;
        const float to_end_x   = pt_x-end_x,   to_end_y   = pt_y-end_y,   to_end_z   =
            pt_z-end_z;

        const float cross_x = to_start_y*to_end_z - to_start_z*to_end_y;
        const float cross_y = to_start_z*to_end_x - to_start_x*to_end_z;

```

```
        const float dist_start = sqrtf(to_start_x*to_start_x + to_start_y*to_start_y + to_start_z*to_start_z);
        const float dist_end   = sqrtf(to_end_x*to_end_x     + to_end_y*to_end_y     + to_end_z*to_end_z);
        const float dot_prod   = to_start_x*to_end_x + to_start_y*to_end_y + to_start_z*to_end_z;
        const float denom      = dist_start * dist_end * (dist_start*dist_end + dot_prod) + 1e-30f;
        const float factor     = (dist_start + dist_end) / denom;

        field_x += cross_x * factor;
        field_y += cross_y * factor;

        const float seg_dx     = end_x-start_x, seg_dy = end_y-start_y, seg_dz = end_z-start_z;
        const float seg_len_sq = seg_dx*seg_dx + seg_dy*seg_dy + seg_dz*seg_dz + 1e-30f;
        float proj_t = ((pt_x-start_x)*seg_dx + (pt_y-start_y)*seg_dy + (pt_z-start_z)*seg_dz) / seg_len_sq;
        proj_t = fminf(1.f, fmaxf(0.f, proj_t));
        const float diff_x  = pt_x - (start_x + proj_t*seg_dx);
        const float diff_y  = pt_y - (start_y + proj_t*seg_dy);
        const float diff_z  = pt_z - (start_z + proj_t*seg_dz);
        const float dist_sq = diff_x*diff_x + diff_y*diff_y + diff_z*diff_z;
        if (dist_sq < min_dist_sq) min_dist_sq = dist_sq;
    }

    const float wire_mask = (sqrtf(min_dist_sq) < mask_radius) ? 0.f : 1.f;
    Bout[point_idx*2]   = prefactor * field_x * wire_mask;
    Bout[point_idx*2+1] = prefactor * field_y * wire_mask;
}
"""

_kernel = None
def _get_kernel():
    global _kernel
    if _kernel is None and HAS_GPU:
        _kernel = cp.RawKernel(_CUDA_SRC, "biot_savart_2d")
    return _kernel

def _warmup_gpu():
    if not HAS_GPU:
        return
    zero_components  = np.zeros(2, dtype=np.float32)
    seg_start_z      = np.array([-0.1, 0.0], dtype=np.float32)
    seg_end_z        = np.array([ 0.0, 0.1], dtype=np.float32)
    dummy_points_gpu = cp.zeros((4, 2), dtype=cp.float32)
    dummy_output_gpu = cp.zeros((4, 2), dtype=cp.float32)
    _get_kernel()((1,), (4,),
        (dummy_points_gpu.ravel(),
         cp.asarray(zero_components), cp.asarray(zero_components), cp.asarray(seg_start_z),
         cp.asarray(zero_components), cp.asarray(zero_components), cp.asarray(seg_end_z),
         dummy_output_gpu.ravel(),
         np.int32(4), np.int32(2), np.float32(1e-7), np.float32(0.04)))
    cp.cuda.Device().synchronize()

def make_segments(wire, num_segments=300):
    if wire == "loop":
        theta     = np.linspace(0, 2*np.pi, num_segments+1, dtype=np.float32)
        x         = LOOP_RADIUS * np.cos(theta)
        y         = LOOP_RADIUS * np.sin(theta)
        z         = np.zeros(num_segments+1, dtype=np.float32)
        seg_start = np.stack([x[:-1], y[:-1], z[:-1]], axis=1)
        seg_end   = np.stack([x[1: ], y[1: ], z[1: ]], axis=1)

    elif wire == "solenoid":
        total_pts = num_segments + 1
        # Parametric helix: t     [0, 2   * N_turns]
        t         = np.linspace(0, 2 * np.pi * SOLENOID_TURNS, total_pts,
            dtype=np.float32)
        x         = SOLENOID_RADIUS * np.cos(t)
        y         = SOLENOID_RADIUS * np.sin(t)
        z         = np.linspace(-SOLENOID_LENGTH / 2, SOLENOID_LENGTH / 2, total_pts,
            dtype=np.float32)
```

```
        seg_start = np.stack([x[:-1], y[:-1], z[:-1]], axis=1)
        seg_end   = np.stack([x[1: ], y[1: ], z[1: ]], axis=1)

    else:  # straight
        z         = np.linspace(-1.5, 1.5, num_segments+1, dtype=np.float32)
        seg_start = np.stack([np.zeros(num_segments), np.zeros(num_segments), z[:-1]],
            axis=1)
        seg_end   = np.stack([np.zeros(num_segments), np.zeros(num_segments), z[1: ]],
            axis=1)

    return seg_start.astype(np.float32), seg_end.astype(np.float32)

# CPU Biot-Savart
def bs_cpu(I, gx, gy, wire="straight", num_segments=300, mask_r=0.04):
    seg_start, seg_end = make_segments(wire, num_segments)
    seg_vec = seg_end - seg_start
    points  = np.stack([gx.ravel(), gy.ravel(), np.zeros(gx.size)], axis=1)

    vec_to_start  = points[:, None, :] - seg_start[None]
    vec_to_end    = points[:, None, :] - seg_end[None]
    cross_product = np.cross(vec_to_start, vec_to_end, axis=2)
    dist_to_start = np.linalg.norm(vec_to_start, axis=2)
    dist_to_end   = np.linalg.norm(vec_to_end,   axis=2)
    dot_product   = np.einsum("ijk,ijk->ij", vec_to_start, vec_to_end)
    denominator   = dist_to_start * dist_to_end * (dist_to_start * dist_to_end +
        dot_product) + 1e-30
    scalar_factor = (dist_to_start + dist_to_end) / denominator
    prefactor     = MU0 * I / (4 * np.pi)
    B_field       = prefactor * np.sum(cross_product * scalar_factor[:, :, None], axis=1)

    seg_len_sq     = np.einsum("ij,ij->i", seg_vec, seg_vec) + 1e-30
    proj_param     = np.einsum("mij,ij->mi", points[:, None, :] - seg_start[None],
        seg_vec) / seg_len_sq
    proj_param     = np.clip(proj_param, 0.0, 1.0)
    nearest_points = seg_start[None] + proj_param[:, :, None] * seg_vec[None]
    dist_sq        = np.sum((points[:, None, :] - nearest_points)**2, axis=2)
    B_field[np.sqrt(np.min(dist_sq, axis=1)) < mask_r] = 0.0

    grid_shape = gx.shape
    return B_field[:, 0].reshape(grid_shape), B_field[:, 1].reshape(grid_shape)

# GPU Biot-Savart
def bs_gpu(I, gx, gy, wire="straight", num_segments=300, mask_r=0.04):
    seg_start, seg_end = make_segments(wire, num_segments)
    num_points = gx.size
    grid_xy    = np.stack([gx.ravel(), gy.ravel()], axis=1).astype(np.float32)

    points_gpu      = cp.ascontiguousarray(cp.asarray(grid_xy))
    field_gpu       = cp.zeros((num_points, 2), dtype=cp.float32)
    seg_start_x_gpu = cp.asarray(seg_start[:, 0])
    seg_start_y_gpu = cp.asarray(seg_start[:, 1])
    seg_start_z_gpu = cp.asarray(seg_start[:, 2])
    seg_end_x_gpu   = cp.asarray(seg_end[:, 0])
    seg_end_y_gpu   = cp.asarray(seg_end[:, 1])
    seg_end_z_gpu   = cp.asarray(seg_end[:, 2])

    prefactor         = np.float32(MU0 * I / (4 * np.pi))
    threads_per_block = 256
    num_blocks        = (num_points + threads_per_block - 1) // threads_per_block

    _get_kernel()(
        (num_blocks,), (threads_per_block,),
        (points_gpu.ravel(),
         seg_start_x_gpu, seg_start_y_gpu, seg_start_z_gpu,
         seg_end_x_gpu,   seg_end_y_gpu,   seg_end_z_gpu,
         field_gpu.ravel(), np.int32(num_points), np.int32(num_segments), prefactor,
             np.float32(mask_r))
    )
    cp.cuda.Device().synchronize()

    field_result = cp.asnumpy(field_gpu)
    grid_shape   = gx.shape
    return field_result[:, 0].reshape(grid_shape), field_result[:, 1].reshape(grid_shape)

```

```
#
#  BENCHMARK MODE
#
def run_benchmark():
    if HAS_GPU:
        print("  [warming up GPU kernel ]")
        _warmup_gpu()

    for wire in BENCH_WIRES:
        print()
        print("=" * 65)
        print(f"  Wire type : {wire.upper()}   S=300 segments")
        print(f"  CPU       : NumPy")
        print(f"  GPU       : {GPU_NAME}")
        print(f"  Runs      : best of {BENCH_REPEATS}")
        print("=" * 65)
        print(f"{'Grid':>6}  {'Points':>8}  {'Ops':>12}  "
              f"{'CPU (ms)':>10}  {'GPU (ms)':>10}  {'Speedup':>8}")
        print("-" * 65)

        for grid_size in BENCH_GRIDS:
            coords     = np.linspace(-EXTENT, EXTENT, grid_size)
            gx, gy     = np.meshgrid(coords, coords)
            num_points = grid_size * grid_size
            num_ops    = num_points * 300

            cpu_times = []
            for _ in range(BENCH_REPEATS):
                t0 = time.perf_counter()
                bs_cpu(1.0, gx, gy, wire=wire)
                cpu_times.append((time.perf_counter() - t0) * 1e3)
            cpu_ms = min(cpu_times)

            if HAS_GPU:
                gpu_times = []
                for _ in range(BENCH_REPEATS):
                    t0 = time.perf_counter()
                    bs_gpu(1.0, gx, gy, wire=wire)
                    gpu_times.append((time.perf_counter() - t0) * 1e3)
                gpu_ms  = min(gpu_times)
                speedup = cpu_ms / gpu_ms
                print(f"{grid_size:>4}    {num_points:>8,}  {num_ops:>12,}  "
                      f"{cpu_ms:>10.1f}  {gpu_ms:>10.1f}  {speedup:>7.1f}x")
            else:
                print(f"{grid_size:>4}    {num_points:>8,}  {num_ops:>12,}  "
                      f"{cpu_ms:>10.1f}  {'N/A':>10}  {'N/A':>8}")

        print("=" * 65)

#
#  GRAPHICAL MODE
#
def run_gui():
    import tkinter as tk
    from tkinter import ttk
    import matplotlib
    matplotlib.use("TkAgg")
    from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
    from matplotlib.figure import Figure
    from matplotlib import cm, colors as mcolors

    def draw_quiver(ax, gx, gy, Bx, By, skip=1, arrow_len=0.10):
        step_slice     = slice(None, None, skip)
        gx_sub, gy_sub = gx[step_slice, step_slice], gy[step_slice, step_slice]
        Bx_sub, By_sub = Bx[step_slice, step_slice], By[step_slice, step_slice]
        B_magnitude    = np.sqrt(Bx_sub**2 + By_sub**2)
        nonzero_B      = B_magnitude[B_magnitude > 0]
        vmin = float(nonzero_B.min())              if len(nonzero_B) else 1e-9
        vmax = float(np.percentile(nonzero_B, 98)) if len(nonzero_B) else 1e-6
        B_mag_safe  = B_magnitude + 1e-30
        log_B_flat  = np.log1p(B_mag_safe.ravel())
        arrow_scale = 0.15 + 0.85*(log_B_flat - log_B_flat.min())/(log_B_flat.max() -
            log_B_flat.min() + 1e-20)
        arrow_scale = arrow_scale.reshape(B_mag_safe.shape)
```

```
        arrow_dx    = Bx_sub / B_mag_safe * arrow_scale * arrow_len
        arrow_dy    = By_sub / B_mag_safe * arrow_scale * arrow_len
        log_norm    = mcolors.LogNorm(vmin=vmin, vmax=vmax)
        arrow_colors = cm.plasma(np.clip(log_norm(B_magnitude.ravel()), 0, 1))
        ax.quiver(gx_sub, gy_sub, arrow_dx, arrow_dy, color=arrow_colors,
                  scale=1.0, scale_units="xy", angles="xy",
                  width=0.003, headwidth=4, headlength=5, alpha=0.92)
        return log_norm, B_magnitude

    root = tk.Tk()
    root.title("2D Biot-Savart Simulator")
    bg_color = "#f4f4f4"
    root.configure(bg=bg_color)

    frame_plot = tk.Frame(root, bg="white")
    frame_plot.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    fig = Figure(figsize=(6, 6), facecolor="white")
    ax  = fig.add_subplot(111, aspect="equal")
    fig.subplots_adjust(left=0.10, right=0.86, bottom=0.08, top=0.92)

    canvas = FigureCanvasTkAgg(fig, master=frame_plot)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
    NavigationToolbar2Tk(canvas, frame_plot).update()

    colorbar_mappable = cm.ScalarMappable(norm=mcolors.Normalize(0, 1), cmap="plasma")
    colorbar_mappable.set_array([])
    cbar = fig.colorbar(colorbar_mappable, ax=ax, fraction=0.03, pad=0.02)
    cbar.set_label("|B| (T)", fontsize=8)

    frame_ctrl = tk.Frame(root, bg=bg_color, width=240)
    frame_ctrl.pack(side=tk.RIGHT, fill=tk.Y, padx=8, pady=8)
    frame_ctrl.pack_propagate(False)

    def section(title):
        label_frame = ttk.LabelFrame(frame_ctrl, text=title, padding="6 4")
        label_frame.pack(fill=tk.X, pady=4)
        return label_frame

    def make_slider(parent, label, var, lo, hi, res):
        row = tk.Frame(parent, bg=bg_color); row.pack(fill=tk.X, pady=2)
        tk.Label(row, text=label, width=12, anchor="w",
                 bg=bg_color, font=("Helvetica", 9)).pack(side=tk.LEFT)
        tk.Label(row, textvariable=var, width=6, anchor="e",
                 bg=bg_color, font=("Helvetica", 9)).pack(side=tk.RIGHT)
        tk.Scale(row, variable=var, from_=lo, to=hi, resolution=res,
                 orient=tk.HORIZONTAL, bg=bg_color, highlightthickness=0,
                 showvalue=False, length=80).pack(side=tk.LEFT, fill=tk.X, expand=True)

    backend_section  = section("Backend")
    gpu_status_color = "#006600" if HAS_GPU else "#990000"
    tk.Label(backend_section, text=f"GPU: {GPU_NAME}",
             fg=gpu_status_color, bg=bg_color, font=("Helvetica", 8)).pack(anchor="w")
    BACKENDS    = ["CPU     NumPy"] + (["GPU     CUDA"] if HAS_GPU else [])
    backend_var = tk.StringVar(value=BACKENDS[0])
    ttk.Combobox(backend_section, textvariable=backend_var, values=BACKENDS,
                 state="readonly", width=22, font=("Helvetica", 9)
                 ).pack(fill=tk.X, pady=(4, 0))

    wire_section = section("Wire Type")
    wire_var = tk.StringVar(value="straight")
    for w in ["straight", "loop", "solenoid"]:
        tk.Radiobutton(wire_section, text=w.capitalize(), variable=wire_var, value=w,
                       bg=bg_color, font=("Helvetica", 9),
                       command=lambda: _debounce()).pack(anchor="w")

    current_section = section("Current")
    I_mag_var = tk.DoubleVar(value=1.0)
    make_slider(current_section, "I  (A)", I_mag_var, 0.5, 20.0, 0.5)
    I_sign = [+1]
    direction_label_var = tk.StringVar(value="Direction:       (out of page)")

    def toggle_dir():
        I_sign[0] *= -1
```

```
        direction_label_var.set("Direction:       (out of page)" if I_sign[0] > 0
                                else "Direction:       (into page)")
        direction_button.config(bg="#e8f0e8" if I_sign[0] > 0 else "#f0e8e8")
        _debounce()

    direction_button = tk.Button(current_section, textvariable=direction_label_var,
        bg="#e8f0e8",
                                 relief="groove", font=("Helvetica", 9),
                                 cursor="hand2", command=toggle_dir)
    direction_button.pack(fill=tk.X, pady=(6, 0))

    computation_section = section("Computation")
    pts_var  = tk.IntVar(value=30)
    skip_var = tk.IntVar(value=1)
    make_slider(computation_section, "Grid pts",   pts_var,  10, 200, 5)
    make_slider(computation_section, "Arrow skip", skip_var,  1,   8, 1)

    timing_section = section("Timing")
    cpu_time_label = tk.Label(timing_section, text="CPU     ", bg=bg_color,
        font=("Courier", 10),
                              fg="#004488", anchor="w")
    cpu_time_label.pack(fill=tk.X)
    gpu_time_label = tk.Label(timing_section, text="GPU     ", bg=bg_color,
        font=("Courier", 10),
                              fg="#006600", anchor="w")
    gpu_time_label.pack(fill=tk.X)
    speedup_label = tk.Label(timing_section, text="", bg=bg_color, font=("Courier", 9),
                             fg="#880000", anchor="w")
    speedup_label.pack(fill=tk.X)
    points_info_label = tk.Label(timing_section, text="", bg=bg_color, font=("Courier",
        8),
                                 fg="#444", anchor="w")
    points_info_label.pack(fill=tk.X)

    last_timing_ms   = {"CPU": None, "GPU": None}
    pending_after_id = [None]

    def _debounce(ms=250):
        if pending_after_id[0]:
            try: root.after_cancel(pending_after_id[0])
            except: pass
        pending_after_id[0] = root.after(ms, refresh)

    def refresh():
        wire_type = wire_var.get()
        I         = float(I_mag_var.get()) * I_sign[0]
        grid_size = int(pts_var.get())
        skip      = int(skip_var.get())
        use_gpu   = backend_var.get().startswith("GPU") and HAS_GPU

        coords = np.linspace(-EXTENT, EXTENT, grid_size)
        gx, gy = np.meshgrid(coords, coords)

        t0 = time.perf_counter()
        if use_gpu:
            Bx, By     = bs_gpu(I, gx, gy, wire=wire_type)
            elapsed_ms = (time.perf_counter() - t0) * 1e3
            last_timing_ms["GPU"] = elapsed_ms
            gpu_time_label.config(text=f"GPU  {elapsed_ms:8.1f} ms")
        else:
            Bx, By     = bs_cpu(I, gx, gy, wire=wire_type)
            elapsed_ms = (time.perf_counter() - t0) * 1e3
            last_timing_ms["CPU"] = elapsed_ms
            cpu_time_label.config(text=f"CPU  {elapsed_ms:8.1f} ms")

        if last_timing_ms["CPU"] and last_timing_ms["GPU"]:
            speedup_label.config(text=f"Speedup
                x{last_timing_ms['CPU']/last_timing_ms['GPU']:.1f}")
        else:
            speedup_label.config(text="Switch backends to compare")

        ax.cla()
        draw_quiver(ax, gx, gy, Bx, By, skip=skip)

```

```
        if wire_type == "loop":
            theta = np.linspace(0, 2*np.pi, 300)
            ax.plot(LOOP_RADIUS*np.cos(theta), LOOP_RADIUS*np.sin(theta),
                    "-", color="red", lw=2.0, alpha=0.85, zorder=5, label="Loop wire")
            ax.legend(fontsize=7, loc="upper right")
        elif wire_type == "solenoid":
            theta = np.linspace(0, 2*np.pi, 300)
            ax.plot(SOLENOID_RADIUS*np.cos(theta), SOLENOID_RADIUS*np.sin(theta),
                    "--", color="red", lw=1.8, alpha=0.85, zorder=5, label="Solenoid (xy
                        cross-section)")
            # mark the turn entry/exit points where the helix pierces z=0
            n_turns = SOLENOID_TURNS
            for k in range(n_turns + 1):
                phi = 2 * np.pi * k
                ax.plot(SOLENOID_RADIUS * np.cos(phi), SOLENOID_RADIUS * np.sin(phi),
                        "o", color="red", ms=5, mec="black", mew=0.6, zorder=6)
            ax.legend(fontsize=7, loc="upper right")
        else:
            wire_symbol = "   " if I_sign[0] > 0 else "   "
            ax.plot(0, 0, "o", color="red", ms=14, mec="black", mew=0.8, zorder=5)
            ax.text(0, 0, wire_symbol, ha="center", va="center",
                    fontsize=14, color="white", zorder=6, fontweight="bold")

        B_magnitude = np.sqrt(Bx**2 + By**2)
        nonzero_B   = B_magnitude[B_magnitude > 0]
        if len(nonzero_B):
            scalar_mappable = cm.ScalarMappable(
                norm=mcolors.LogNorm(vmin=float(nonzero_B.min()),
                                     vmax=float(np.percentile(nonzero_B, 98))),
                cmap="plasma")
            scalar_mappable.set_array([])
            cbar.update_normal(scalar_mappable)
        cbar.set_label("|B| (T)", fontsize=8)

        ax.set_xlim(-EXTENT, EXTENT); ax.set_ylim(-EXTENT, EXTENT)
        ax.set_xlabel("x [m]", fontsize=9); ax.set_ylabel("y [m]", fontsize=9)
        wire_label      = (f"Loop R={LOOP_RADIUS}m" if wire_type == "loop" else
                           f"Solenoid R={SOLENOID_RADIUS}m N={SOLENOID_TURNS}" if
                               wire_type == "solenoid" else
                           "Straight wire")
        backend_label   = "GPU" if use_gpu else "CPU"
        direction_label = "   out" if I_sign[0] > 0 else "    in"
        ax.set_title(f"{wire_label}   I={I:+.1f}A ({direction_label})
            {grid_size}  {grid_size}   [{backend_label}]",
                     fontsize=9)
        ax.set_facecolor("#fafafa")
        ax.grid(True, color="#ddd", linewidth=0.4)
        ax.set_aspect("equal")

        num_points = grid_size * grid_size
        points_info_label.config(text=f"{num_points:,} pts    300 segs =
            {num_points*300:,} ops")
        canvas.draw_idle()

    for v in (backend_var, wire_var, I_mag_var, pts_var, skip_var):
        v.trace_add("write", lambda *_: _debounce())

    if HAS_GPU:
        _warmup_gpu()
    refresh()
    root.mainloop()

if __name__ == "__main__":
    if BENCHMARK:
        run_benchmark()
    else:
        run_gui()
```

Listing 1: Code voor 2D simulaties

B Code voor de 3D-weergave

```
import numpy as np
import time



BENCHMARK = False



BENCH_GRIDS   = [10, 20, 30, 50, 75, 100, 125, 150, 175, 200]
BENCH_REPEATS = 3
BENCH_WIRES   = ["straight", "loop", "solenoid"]



MU0              = 4e-7 * np.pi
EXTENT           = 1.0
LOOP_RADIUS      = 0.5
SOLENOID_RADIUS  = 0.35
SOLENOID_LENGTH  = 1.2
SOLENOID_TURNS   = 4



try:
    import cupy as cp
    cp.cuda.Device(0).use()
    _ = cp.array([1.0], dtype=cp.float32)
    HAS_GPU   = True
    gpu_props = cp.cuda.runtime.getDeviceProperties(0)
    GPU_NAME  = gpu_props["name"].decode() if isinstance(gpu_props["name"], bytes) else
        str(gpu_props["name"])
except Exception:
    HAS_GPU  = False
    cp       = None
    GPU_NAME = "Not available"



_CUDA_SRC = r"""
extern "C" __global__
void biot_savart_3d(
    const float* __restrict__ pts,
    const float* __restrict__ Ax,
    const float* __restrict__ Ay,
    const float* __restrict__ Az,
    const float* __restrict__ Bx,
    const float* __restrict__ By,
    const float* __restrict__ Bz,
    float*       __restrict__ Bout,
    const int num_points, const int num_segments,
    const float prefactor, const float mask_radius)
{
    int point_idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (point_idx >= num_points) return;

    const float pt_x = pts[point_idx*3];
    const float pt_y = pts[point_idx*3+1];
    const float pt_z = pts[point_idx*3+2];
    float field_x = 0.f, field_y = 0.f, field_z = 0.f;
    float min_dist_sq = 1e30f;

    for (int k = 0; k < num_segments; ++k) {
        const float start_x = Ax[k], start_y = Ay[k], start_z = Az[k];
        const float end_x   = Bx[k], end_y   = By[k], end_z   = Bz[k];

        const float to_start_x = pt_x-start_x, to_start_y = pt_y-start_y, to_start_z =
            pt_z-start_z;
        const float to_end_x   = pt_x-end_x,   to_end_y   = pt_y-end_y,   to_end_z   =
            pt_z-end_z;

        const float cross_x = to_start_y*to_end_z - to_start_z*to_end_y;
        const float cross_y = to_start_z*to_end_x - to_start_x*to_end_z;
```

```
        const float cross_z = to_start_x*to_end_y - to_start_y*to_end_x;

        const float dist_start = sqrtf(to_start_x*to_start_x + to_start_y*to_start_y +
            to_start_z*to_start_z);
        const float dist_end   = sqrtf(to_end_x*to_end_x     + to_end_y*to_end_y     +
            to_end_z*to_end_z);
        const float dot_prod   = to_start_x*to_end_x + to_start_y*to_end_y +
            to_start_z*to_end_z;
        const float denom      = dist_start * dist_end * (dist_start*dist_end + dot_prod)
            + 1e-30f;
        const float factor     = (dist_start + dist_end) / denom;

        field_x += cross_x * factor;
        field_y += cross_y * factor;
        field_z += cross_z * factor;

        const float seg_dx     = end_x-start_x, seg_dy = end_y-start_y, seg_dz =
            end_z-start_z;
        const float seg_len_sq = seg_dx*seg_dx + seg_dy*seg_dy + seg_dz*seg_dz + 1e-30f;
        float proj_t = ((pt_x-start_x)*seg_dx + (pt_y-start_y)*seg_dy +
            (pt_z-start_z)*seg_dz) / seg_len_sq;
        proj_t = fminf(1.f, fmaxf(0.f, proj_t));
        const float diff_x  = pt_x - (start_x + proj_t*seg_dx);
        const float diff_y  = pt_y - (start_y + proj_t*seg_dy);
        const float diff_z  = pt_z - (start_z + proj_t*seg_dz);
        const float dist_sq = diff_x*diff_x + diff_y*diff_y + diff_z*diff_z;
        if (dist_sq < min_dist_sq) min_dist_sq = dist_sq;
    }

    const float wire_mask = (sqrtf(min_dist_sq) < mask_radius) ? 0.f : 1.f;
    Bout[point_idx*3]   = prefactor * field_x * wire_mask;
    Bout[point_idx*3+1] = prefactor * field_y * wire_mask;
    Bout[point_idx*3+2] = prefactor * field_z * wire_mask;
}
"""



_kernel = None
def _get_kernel():
    global _kernel
    if _kernel is None and HAS_GPU:
        _kernel = cp.RawKernel(_CUDA_SRC, "biot_savart_3d")
    return _kernel



def _warmup_gpu():
    if not HAS_GPU:
        return
    zero_components  = np.zeros(2, dtype=np.float32)
    seg_start_z      = np.array([-0.1, 0.0], dtype=np.float32)
    seg_end_z        = np.array([ 0.0, 0.1], dtype=np.float32)
    dummy_points_gpu = cp.zeros((4, 3), dtype=cp.float32)
    dummy_output_gpu = cp.zeros((4, 3), dtype=cp.float32)
    _get_kernel()((1,), (4,),
        (dummy_points_gpu.ravel(),
         cp.asarray(zero_components), cp.asarray(zero_components),
             cp.asarray(seg_start_z),
         cp.asarray(zero_components), cp.asarray(zero_components), cp.asarray(seg_end_z),
         dummy_output_gpu.ravel(),
         np.int32(4), np.int32(2), np.float32(1e-7), np.float32(0.04)))
    cp.cuda.Device().synchronize()



TWO_WIRE_TYPES = ("parallel_same", "parallel_opp", "cross")


def make_segments(wire_type, num_segments=300):
    """Generate (seg_start, seg_end) float32 arrays for the given wire type."""

    if wire_type == "straight":
        z   = np.linspace(-1.5, 1.5, num_segments + 1, dtype=np.float32)
```

```
        pts = np.stack([np.zeros(num_segments + 1, dtype=np.float32),
                        np.zeros(num_segments + 1, dtype=np.float32), z], axis=1)
        return pts[:-1], pts[1:]

    elif wire_type == "loop":
        t   = np.linspace(0, 2 * np.pi, num_segments + 1, dtype=np.float32)
        x   = LOOP_RADIUS * np.cos(t)
        y   = LOOP_RADIUS * np.sin(t)
        z   = np.zeros(num_segments + 1, dtype=np.float32)
        pts = np.stack([x, y, z], axis=1)
        return pts[:-1], pts[1:]

    elif wire_type == "solenoid":
        t   = np.linspace(0, 2 * np.pi * SOLENOID_TURNS, num_segments + 1,
            dtype=np.float32)
        x   = SOLENOID_RADIUS * np.cos(t)
        y   = SOLENOID_RADIUS * np.sin(t)
        z   = np.linspace(-SOLENOID_LENGTH / 2, SOLENOID_LENGTH / 2,
                          num_segments + 1, dtype=np.float32)
        pts = np.stack([x, y, z], axis=1)
        return pts[:-1], pts[1:]

    elif wire_type == "parallel_same":
        half = num_segments // 2
        rest = num_segments - half
        z1   = np.linspace(-1.5, 1.5, half + 1, dtype=np.float32)
        z2   = np.linspace(-1.5, 1.5, rest + 1, dtype=np.float32)
        pts1 = np.stack([np.full(half + 1, -0.3, dtype=np.float32),
                         np.zeros(half + 1, dtype=np.float32), z1], axis=1)
        pts2 = np.stack([np.full(rest + 1,  0.3, dtype=np.float32),
                         np.zeros(rest + 1, dtype=np.float32), z2], axis=1)
        return (np.vstack([pts1[:-1], pts2[:-1]]),
                np.vstack([pts1[1:],  pts2[1:]]))

    elif wire_type == "parallel_opp":
        half = num_segments // 2
        rest = num_segments - half
        z1   = np.linspace(-1.5,  1.5, half + 1, dtype=np.float32)
        z2   = np.linspace( 1.5, -1.5, rest + 1, dtype=np.float32)   # reversed current
        pts1 = np.stack([np.full(half + 1, -0.3, dtype=np.float32),
                         np.zeros(half + 1, dtype=np.float32), z1], axis=1)
        pts2 = np.stack([np.full(rest + 1,  0.3, dtype=np.float32),
                         np.zeros(rest + 1, dtype=np.float32), z2], axis=1)
        return (np.vstack([pts1[:-1], pts2[:-1]]),
                np.vstack([pts1[1:],  pts2[1:]]))

    elif wire_type == "cross":
        half = num_segments // 2
        rest = num_segments - half
        z    = np.linspace(-1.5, 1.5, half + 1, dtype=np.float32)
        x    = np.linspace(-1.5, 1.5, rest + 1, dtype=np.float32)
        pts1 = np.stack([np.zeros(half + 1, dtype=np.float32),
                         np.zeros(half + 1, dtype=np.float32), z], axis=1)
        pts2 = np.stack([x,
                         np.full(rest + 1, 0.05, dtype=np.float32),
                         np.zeros(rest + 1, dtype=np.float32)], axis=1)
        return (np.vstack([pts1[:-1], pts2[:-1]]),
                np.vstack([pts1[1:],  pts2[1:]]))

    else:
        raise ValueError(f"Unknown wire type: {wire_type!r}")


def _wire_subsegments(wire_type, seg_start, seg_end, num_segments):
    """Split concatenated segment arrays back into per-wire lists for correct masking."""
    if wire_type in TWO_WIRE_TYPES:
        half = num_segments // 2
        rest = num_segments - half
        return (
            [seg_start[:half],  seg_start[half:]],
            [seg_end[:half],    seg_end[half:]],
            [half, rest],
        )
```

```
    return [seg_start], [seg_end], [num_segments]



def _make_grid(grid_size, plane="XY", offset=0.0):
    coords = np.linspace(-EXTENT, EXTENT, grid_size, dtype=np.float32)
    if plane == "XY":
        gA, gB = np.meshgrid(coords, coords)
        gx, gy, gz = gA, gB, np.full_like(gA, offset)
    elif plane == "XZ":
        gA, gB = np.meshgrid(coords, coords)
        gx, gy, gz = gA, np.full_like(gA, offset), gB
    else:
        gA, gB = np.meshgrid(coords, coords)
        gx, gy, gz = np.full_like(gA, offset), gA, gB
    return gx, gy, gz



def _make_volume_grid(grid_size):
    coords = np.linspace(-EXTENT, EXTENT, grid_size, dtype=np.float32)
    gx, gy, gz = np.meshgrid(coords, coords, coords)
    return gx, gy, gz



def bs_cpu(I, gx, gy, gz, wire="straight", num_segments=300, mask_r=0.04):
    seg_start, seg_end = make_segments(wire, num_segments)
    starts_list, ends_list, counts_list = _wire_subsegments(wire, seg_start, seg_end,
        num_segments)

    points     = np.stack([gx.ravel(), gy.ravel(), gz.ravel()], axis=1).astype(np.float64)
    grid_shape = gx.shape
    B_total    = np.zeros((points.shape[0], 3), dtype=np.float64)
    prefactor  = MU0 * I / (4 * np.pi)

    for w_start, w_end in zip(starts_list, ends_list):
        w_start = w_start.astype(np.float64)
        w_end   = w_end.astype(np.float64)
        seg_vec = w_end - w_start

        vec_to_start  = points[:, None, :] - w_start[None]
        vec_to_end    = points[:, None, :] - w_end[None]
        cross_product = np.cross(vec_to_start, vec_to_end, axis=2)
        dist_to_start = np.linalg.norm(vec_to_start, axis=2)
        dist_to_end   = np.linalg.norm(vec_to_end,   axis=2)
        dot_product   = np.einsum("ijk,ijk->ij", vec_to_start, vec_to_end)
        denominator   = dist_to_start * dist_to_end * (dist_to_start * dist_to_end +
            dot_product) + 1e-30
        scalar_factor = (dist_to_start + dist_to_end) / denominator
        B_wire        = prefactor * np.sum(cross_product * scalar_factor[:, :, None],
            axis=1)

        seg_len_sq     = np.einsum("ij,ij->i", seg_vec, seg_vec) + 1e-30
        proj_param     = np.einsum("mij,ij->mi",
                               points[:, None, :] - w_start[None], seg_vec) / seg_len_sq
        proj_param     = np.clip(proj_param, 0.0, 1.0)
        nearest_points = w_start[None] + proj_param[:, :, None] * seg_vec[None]
        dist_sq        = np.sum((points[:, None, :] - nearest_points) ** 2, axis=2)
        B_wire[np.sqrt(np.min(dist_sq, axis=1)) < mask_r] = 0.0

        B_total += B_wire

    return (B_total[:, 0].reshape(grid_shape),
            B_total[:, 1].reshape(grid_shape),
            B_total[:, 2].reshape(grid_shape))



def bs_gpu(I, gx, gy, gz, wire="straight", num_segments=300, mask_r=0.04):
    seg_start, seg_end = make_segments(wire, num_segments)
    starts_list, ends_list, counts_list = _wire_subsegments(wire, seg_start, seg_end,
        num_segments)

```

```
    num_points = gx.size
    grid_xyz   = np.stack([gx.ravel(), gy.ravel(), gz.ravel()], axis=1).astype(np.float32)
    points_gpu = cp.ascontiguousarray(cp.asarray(grid_xyz))
    prefactor  = np.float32(MU0 * I / (4 * np.pi))

    threads_per_block = 256
    num_blocks        = (num_points + threads_per_block - 1) // threads_per_block
    field_total       = cp.zeros((num_points, 3), dtype=cp.float32)

    for w_start, w_end, w_count in zip(starts_list, ends_list, counts_list):
        field_gpu       = cp.zeros((num_points, 3), dtype=cp.float32)
        seg_start_x_gpu = cp.asarray(w_start[:, 0])
        seg_start_y_gpu = cp.asarray(w_start[:, 1])
        seg_start_z_gpu = cp.asarray(w_start[:, 2])
        seg_end_x_gpu   = cp.asarray(w_end[:, 0])
        seg_end_y_gpu   = cp.asarray(w_end[:, 1])
        seg_end_z_gpu   = cp.asarray(w_end[:, 2])

        _get_kernel()(
            (num_blocks,), (threads_per_block,),
            (points_gpu.ravel(),
             seg_start_x_gpu, seg_start_y_gpu, seg_start_z_gpu,
             seg_end_x_gpu,   seg_end_y_gpu,   seg_end_z_gpu,
             field_gpu.ravel(),
             np.int32(num_points), np.int32(w_count),
             prefactor, np.float32(mask_r))
        )
        cp.cuda.Device().synchronize()
        field_total += field_gpu

    field_result = cp.asnumpy(field_total)
    grid_shape   = gx.shape
    return (field_result[:, 0].reshape(grid_shape),
            field_result[:, 1].reshape(grid_shape),
            field_result[:, 2].reshape(grid_shape))



def run_benchmark():
    if HAS_GPU:
        print("  [warming up GPU kernel...]")
        _warmup_gpu()

    for wire in BENCH_WIRES:
        print()
        print("=" * 65)
        print(f"  Wire type : {wire.upper()}   S=300 segments")
        print(f"  CPU       : NumPy")
        print(f"  GPU       : {GPU_NAME}")
        print(f"  Runs      : best of {BENCH_REPEATS}")
        print("=" * 65)
        print(f"{'Grid':>6}  {'Points':>8}  {'Ops':>12}  "
              f"{'CPU (ms)':>10}  {'GPU (ms)':>10}  {'Speedup':>8}")
        print("-" * 65)

        for grid_size in BENCH_GRIDS:
            gx, gy, gz = _make_grid(grid_size, plane="XY")
            num_points  = grid_size * grid_size
            num_ops     = num_points * 300

            cpu_times = []
            for _ in range(BENCH_REPEATS):
                t0 = time.perf_counter()
                bs_cpu(1.0, gx, gy, gz, wire=wire)
                cpu_times.append((time.perf_counter() - t0) * 1e3)
            cpu_ms = min(cpu_times)

            if HAS_GPU:
                gpu_times = []
                for _ in range(BENCH_REPEATS):
                    t0 = time.perf_counter()
                    bs_gpu(1.0, gx, gy, gz, wire=wire)
                    gpu_times.append((time.perf_counter() - t0) * 1e3)
                gpu_ms  = min(gpu_times)
```

```
                speedup = cpu_ms / gpu_ms
                print(f"{grid_size:>4}    {num_points:>8,}  {num_ops:>12,}  "
                      f"{cpu_ms:>10.1f}  {gpu_ms:>10.1f}  {speedup:>7.1f}x")
            else:
                print(f"{grid_size:>4}    {num_points:>8,}  {num_ops:>12,}  "
                      f"{cpu_ms:>10.1f}  {'N/A':>10}  {'N/A':>8}")

        print("=" * 65)



def run_gui():
    import tkinter as tk
    from tkinter import ttk
    import matplotlib
    matplotlib.use("TkAgg")
    from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg, NavigationToolbar2Tk
    from matplotlib.figure import Figure
    from mpl_toolkits.mplot3d import Axes3D
    from matplotlib import cm, colors as mcolors


    def draw_quiver_3d(ax, gx, gy, gz, Bx, By, Bz, skip, arrow_len, log_norm):
        s  = slice(None, None, skip)
        x  = gx.ravel()[::skip] if gx.ndim == 3 else gx[s, s].ravel()
        y  = gy.ravel()[::skip] if gy.ndim == 3 else gy[s, s].ravel()
        z  = gz.ravel()[::skip] if gz.ndim == 3 else gz[s, s].ravel()
        bx = Bx.ravel()[::skip] if Bx.ndim == 3 else Bx[s, s].ravel()
        by = By.ravel()[::skip] if By.ndim == 3 else By[s, s].ravel()
        bz = Bz.ravel()[::skip] if Bz.ndim == 3 else Bz[s, s].ravel()
        B_mag  = np.sqrt(bx**2 + by**2 + bz**2)
        B_safe = B_mag + 1e-30
        log_flat = np.log1p(B_safe)
        scale    = 0.15 + 0.85 * (log_flat - log_flat.min()) / (log_flat.max() -
            log_flat.min() + 1e-20)
        dx = bx / B_safe * scale * arrow_len
        dy = by / B_safe * scale * arrow_len
        dz = bz / B_safe * scale * arrow_len
        colors_arr = cm.plasma(np.clip(log_norm(B_mag), 0, 1))
        ax.quiver(x, y, z, dx, dy, dz,
                  colors=colors_arr, length=1.0, normalize=False,
                  arrow_length_ratio=0.3, linewidth=0.8, alpha=0.88)


    def draw_wire_3d(ax, wire_type, I_sign):
        t = np.linspace(0, 2 * np.pi, 300)
        if wire_type == "loop":
            ax.plot(LOOP_RADIUS * np.cos(t), LOOP_RADIUS * np.sin(t),
                    np.zeros(300), "-", color="red", lw=2.0, alpha=0.9, zorder=5)
        elif wire_type == "solenoid":
            ts = np.linspace(0, 2 * np.pi * SOLENOID_TURNS, 600)
            ax.plot(SOLENOID_RADIUS * np.cos(ts), SOLENOID_RADIUS * np.sin(ts),
                    np.linspace(-SOLENOID_LENGTH / 2, SOLENOID_LENGTH / 2, 600),
                    "-", color="red", lw=2.0, alpha=0.9, zorder=5)
        elif wire_type in ("parallel_same", "parallel_opp"):
            z_ends = [-1.5, 1.5]
            ax.plot([-0.3, -0.3], [0, 0], z_ends, "-", color="red", lw=2.5, alpha=0.9,
                zorder=5)
            color2 = "blue" if wire_type == "parallel_opp" else "red"
            ax.plot([ 0.3,  0.3], [0, 0],
                    z_ends if wire_type == "parallel_same" else z_ends[::-1],
                    "-", color=color2, lw=2.5, alpha=0.9, zorder=5)
            sym1 = "\u2299" if I_sign > 0 else "\u2297"
            sym2 = sym1 if wire_type == "parallel_same" else ("\u2297" if I_sign > 0 else
                "\u2299")
            ax.text(-0.3, 0, 1.6, sym1, color="red",   fontsize=9, ha="center")
            ax.text( 0.3, 0, 1.6, sym2, color=color2,  fontsize=9, ha="center")
        elif wire_type == "cross":
            ax.plot([0, 0],       [0, 0],       [-1.5, 1.5], "-", color="red",  lw=2.5,
                alpha=0.9, zorder=5)
            ax.plot([-1.5, 1.5],  [0.05, 0.05], [0, 0],      "-", color="blue", lw=2.5,
                alpha=0.9, zorder=5)
            sym = "\u2299" if I_sign > 0 else "\u2297"
            ax.text(0,    0.0,  1.6, sym,      color="red",  fontsize=9,  ha="center")
```

```
            ax.text(1.65, 0.05, 0.0, "\u2192", color="blue", fontsize=10, ha="left")
        else:  # straight
            sym = "\u2299" if I_sign > 0 else "\u2297"
            ax.plot([0, 0], [0, 0], [-1.5, 1.5], "-", color="red", lw=2.5, alpha=0.9,
                zorder=5)
            ax.text(0, 0, 1.55, sym, color="red", fontsize=10, ha="center", va="bottom")


    root = tk.Tk()
    root.title("3D Biot-Savart Simulator")
    bg_color = "#f4f4f4"
    root.configure(bg=bg_color)

    frame_plot = tk.Frame(root, bg="white")
    frame_plot.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    fig = Figure(figsize=(7, 7), facecolor="white")
    ax  = fig.add_subplot(111, projection="3d")
    fig.subplots_adjust(left=0.02, right=0.88, bottom=0.02, top=0.96)

    canvas = FigureCanvasTkAgg(fig, master=frame_plot)
    canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
    NavigationToolbar2Tk(canvas, frame_plot).update()

    colorbar_mappable = cm.ScalarMappable(norm=mcolors.Normalize(0, 1), cmap="plasma")
    colorbar_mappable.set_array([])
    cbar = fig.colorbar(colorbar_mappable, ax=ax, fraction=0.025, pad=0.04, shrink=0.6)
    cbar.set_label("|B| (T)", fontsize=8)

    frame_ctrl = tk.Frame(root, bg=bg_color, width=240)
    frame_ctrl.pack(side=tk.RIGHT, fill=tk.Y, padx=8, pady=8)
    frame_ctrl.pack_propagate(False)

    def section(title):
        lf = ttk.LabelFrame(frame_ctrl, text=title, padding="6 4")
        lf.pack(fill=tk.X, pady=4)
        return lf

    def make_slider(parent, label, var, lo, hi, res):
        row = tk.Frame(parent, bg=bg_color)
        row.pack(fill=tk.X, pady=2)
        tk.Label(row, text=label, width=12, anchor="w",
                 bg=bg_color, font=("Helvetica", 9)).pack(side=tk.LEFT)
        tk.Label(row, textvariable=var, width=6, anchor="e",
                 bg=bg_color, font=("Helvetica", 9)).pack(side=tk.RIGHT)
        tk.Scale(row, variable=var, from_=lo, to=hi, resolution=res,
                 orient=tk.HORIZONTAL, bg=bg_color, highlightthickness=0,
                 showvalue=False, length=80).pack(side=tk.LEFT, fill=tk.X, expand=True)

    backend_section  = section("Backend")
    gpu_status_color = "#006600" if HAS_GPU else "#990000"
    tk.Label(backend_section, text=f"GPU: {GPU_NAME}",
             fg=gpu_status_color, bg=bg_color, font=("Helvetica", 8)).pack(anchor="w")
    BACKENDS    = ["CPU \u2013 NumPy"] + (["GPU \u2013 CUDA"] if HAS_GPU else [])
    backend_var = tk.StringVar(value=BACKENDS[0])
    ttk.Combobox(backend_section, textvariable=backend_var, values=BACKENDS,
                 state="readonly", width=22, font=("Helvetica", 9)).pack(fill=tk.X,
                     pady=(4, 0))

    wire_section = section("Wire Type")
    wire_var     = tk.StringVar(value="straight")
    WIRE_OPTIONS = [
        ("straight",      "Straight wire"),
        ("loop",          "Loop"),
        ("solenoid",      "Solenoid"),
        ("parallel_same", "Parallel \u2191\u2191"),
        ("parallel_opp",  "Parallel \u2191\u2193"),
        ("cross",         "Cross wire (\u22a5)"),
    ]
    for val, label in WIRE_OPTIONS:
        tk.Radiobutton(wire_section, text=label, variable=wire_var, value=val,
                       bg=bg_color, font=("Helvetica", 9),
                       command=lambda: _debounce()).pack(anchor="w")

```

```
    mode_section  = section("View Mode")
    mode_var      = tk.StringVar(value="slice")
    plane_var     = tk.StringVar(value="XY")
    offset_var    = tk.DoubleVar(value=0.0)
    vol_pts_var   = tk.IntVar(value=8)

    tk.Radiobutton(mode_section, text="Single slice", variable=mode_var, value="slice",
                   bg=bg_color, font=("Helvetica", 9),
                   command=lambda: _on_mode_change()).pack(anchor="w")
    tk.Radiobutton(mode_section, text="Full 3D volume", variable=mode_var, value="volume",
                   bg=bg_color, font=("Helvetica", 9),
                   command=lambda: _on_mode_change()).pack(anchor="w")

    slice_frame = tk.Frame(mode_section, bg=bg_color)
    slice_frame.pack(fill=tk.X, pady=(4, 0))
    tk.Label(slice_frame, text="Plane:", bg=bg_color,
             font=("Helvetica", 9)).pack(side=tk.LEFT)
    for p in ["XY", "XZ", "YZ"]:
        tk.Radiobutton(slice_frame, text=p, variable=plane_var, value=p,
                       bg=bg_color, font=("Helvetica", 9),
                       command=lambda: _on_plane_change()).pack(side=tk.LEFT)

    offset_frame = tk.Frame(mode_section, bg=bg_color)
    offset_frame.pack(fill=tk.X)
    make_slider(offset_frame, "Plane pos", offset_var, -EXTENT, EXTENT, 0.05)

    vol_frame = tk.Frame(mode_section, bg=bg_color)
    vol_frame.pack(fill=tk.X, pady=(4, 0))
    make_slider(vol_frame, "Vol grid", vol_pts_var, 3, 20, 1)
    vol_warn = tk.Label(vol_frame, text="", bg=bg_color, font=("Helvetica", 8),
                        fg="#993300", wraplength=200, justify="left")
    vol_warn.pack(anchor="w")

    current_section = section("Current")
    I_mag_var       = tk.DoubleVar(value=1.0)
    make_slider(current_section, "I  (A)", I_mag_var, 0.5, 20.0, 0.5)
    I_sign = [+1]
    direction_label_var = tk.StringVar(value="Direction:  \u2191")

    def toggle_dir():
        I_sign[0] *= -1
        direction_label_var.set("Direction:  \u2191" if I_sign[0] > 0
                                else "Direction:  \u2193")
        direction_button.config(bg="#e8f0e8" if I_sign[0] > 0 else "#f0e8e8")
        _debounce()

    direction_button = tk.Button(current_section, textvariable=direction_label_var,
                                 bg="#e8f0e8", relief="groove", font=("Helvetica", 9),
                                 cursor="hand2", command=toggle_dir)
    direction_button.pack(fill=tk.X, pady=(6, 0))

    computation_section = section("Computation")
    pts_var  = tk.IntVar(value=15)
    skip_var = tk.IntVar(value=1)
    make_slider(computation_section, "Grid pts",   pts_var,  5, 50, 5)
    make_slider(computation_section, "Arrow skip", skip_var, 1,  8, 1)

    timing_section = section("Timing")
    cpu_time_label = tk.Label(timing_section, text="CPU  \u2014", bg=bg_color,
                              font=("Courier", 10), fg="#004488", anchor="w")
    cpu_time_label.pack(fill=tk.X)
    gpu_time_label = tk.Label(timing_section, text="GPU  \u2014", bg=bg_color,
                              font=("Courier", 10), fg="#006600", anchor="w")
    gpu_time_label.pack(fill=tk.X)
    speedup_label = tk.Label(timing_section, text="", bg=bg_color, font=("Courier", 9),
                             fg="#880000", anchor="w")
    speedup_label.pack(fill=tk.X)
    points_info_label = tk.Label(timing_section, text="", bg=bg_color, font=("Courier",
        8),
                                 fg="#444", anchor="w")
    points_info_label.pack(fill=tk.X)

    last_timing_ms   = {"CPU": None, "GPU": None}
    pending_after_id = [None]
```

```
569
570     def _debounce(ms=250):
571         if pending_after_id[0]:
572             try: root.after_cancel(pending_after_id[0])
573             except: pass
574         pending_after_id[0] = root.after(ms, refresh)
575
576     def _on_plane_change():
577         offset_var.set(0.0)
578         _debounce()
579
580     def _on_mode_change():
581         n = int(vol_pts_var.get())
582         if mode_var.get() == "volume":
583             vol_warn.config(text=f"{n**3:,} pts \u2014 use small grids (\u226412) on CPU")
584         else:
585             vol_warn.config(text="")
586         _debounce()
587
588     def _update_vol_warn(*_):
589         if mode_var.get() == "volume":
590             n = int(vol_pts_var.get())
591             vol_warn.config(text=f"{n**3:,} pts \u2014 use small grids (\u226412) on CPU")
592
593     def refresh():
594         wire_type = wire_var.get()
595         plane     = plane_var.get()
596         I         = float(I_mag_var.get()) * I_sign[0]
597         grid_size = int(pts_var.get())
598         skip      = int(skip_var.get())
599         offset    = float(offset_var.get())
600         use_gpu   = backend_var.get().startswith("GPU") and HAS_GPU
601         mode      = mode_var.get()
602         vol_n     = int(vol_pts_var.get())
603
604         t0 = time.perf_counter()
605
606         if mode == "volume":
607             gx, gy, gz = _make_volume_grid(vol_n)
608             if use_gpu:
609                 Bx, By, Bz = bs_gpu(I, gx, gy, gz, wire=wire_type)
610             else:
611                 Bx, By, Bz = bs_cpu(I, gx, gy, gz, wire=wire_type)
612             all_grids  = [(gx, gy, gz, Bx, By, Bz)]
613             num_points = vol_n ** 3
614
615         else:
616             gx, gy, gz = _make_grid(grid_size, plane=plane, offset=offset)
617             if use_gpu:
618                 Bx, By, Bz = bs_gpu(I, gx, gy, gz, wire=wire_type)
619             else:
620                 Bx, By, Bz = bs_cpu(I, gx, gy, gz, wire=wire_type)
621             all_grids  = [(gx, gy, gz, Bx, By, Bz)]
622             num_points = grid_size * grid_size
623
624         elapsed_ms = (time.perf_counter() - t0) * 1e3
625         if use_gpu:
626             last_timing_ms["GPU"] = elapsed_ms
627             gpu_time_label.config(text=f"GPU  {elapsed_ms:8.1f} ms")
628         else:
629             last_timing_ms["CPU"] = elapsed_ms
630             cpu_time_label.config(text=f"CPU  {elapsed_ms:8.1f} ms")
631
632         if last_timing_ms["CPU"] and last_timing_ms["GPU"]:
633             speedup_label.config(text=f"Speedup
                x{last_timing_ms['CPU']/last_timing_ms['GPU']:.1f}")
634         else:
635             speedup_label.config(text="Switch backends to compare")
636
637         all_nonzero = np.concatenate([
638             np.sqrt(Bx**2 + By**2 + Bz**2).ravel()
639             for (_, _, _, Bx, By, Bz) in all_grids
640         ])
641         all_nonzero = all_nonzero[all_nonzero > 0]
```

```
        vmin     = float(all_nonzero.min())      if len(all_nonzero) else 1e-9
        vmax     = float(np.percentile(all_nonzero, 98)) if len(all_nonzero) else 1e-6
        log_norm = mcolors.LogNorm(vmin=max(vmin, 1e-30), vmax=max(vmax, 1e-29))

        ax.cla()
        for (gx, gy, gz, Bx, By, Bz) in all_grids:
            draw_quiver_3d(ax, gx, gy, gz, Bx, By, Bz, skip, 0.10, log_norm)

        draw_wire_3d(ax, wire_type, I_sign[0])

        scalar_mappable = cm.ScalarMappable(norm=log_norm, cmap="plasma")
        scalar_mappable.set_array([])
        cbar.update_normal(scalar_mappable)
        cbar.set_label("|B| (T)", fontsize=8)

        ax.set_xlim(-EXTENT, EXTENT)
        ax.set_ylim(-EXTENT, EXTENT)
        ax.set_zlim(-EXTENT, EXTENT)
        ax.set_xlabel("x [m]", fontsize=8)
        ax.set_ylabel("y [m]", fontsize=8)
        ax.set_zlabel("z [m]", fontsize=8)
        ax.tick_params(labelsize=7)

        wire_label = (f"Loop R={LOOP_RADIUS}m"        if wire_type == "loop"     else
                      f"Solenoid R={SOLENOID_RADIUS}m N={SOLENOID_TURNS}" if wire_type ==
                          "solenoid" else
                      wire_type.replace("_", " ").title())
        backend_label = "GPU" if use_gpu else "CPU"
        off_str    = f" offset={offset:+.2f}m" if abs(offset) > 1e-3 else ""
        mode_label = f"3D volume {vol_n}\u00b3" if mode == "volume" else
            f"{plane}{off_str}"
        ax.set_title(f"{wire_label}   I={I:+.1f}A   {mode_label}   [{backend_label}]",
            fontsize=9)

        points_info_label.config(
            text=f"{num_points:,} pts \u00d7 300 segs = {num_points*300:,} ops")
        canvas.draw_idle()

    for v in (backend_var, wire_var, I_mag_var, pts_var, skip_var, offset_var,
        vol_pts_var):
        v.trace_add("write", lambda *_: _debounce())

    vol_pts_var.trace_add("write", _update_vol_warn)

    if HAS_GPU:
        _warmup_gpu()
    refresh()
    root.mainloop()



if __name__ == "__main__":
    if BENCHMARK:
        run_benchmark()
    else:
        run_gui()
```

Listing 2: Finale code voor de 3D simulaties

Referenties

[1] A. V. d. Stede, "ArneVanderStede/MagneticFieldSimulator," Apr. 2026. original-date: 2026-04-30T11:01:22Z.

[2] MIT Department of Physics, "IX. Source of Magnetic Fields – Worked Examples," Course Notes 8.02, Massachusetts Institute of Technology, 2003. Course 8.02, Spring 2003. Available at `https://web.mit.edu/8.02-esg/Spring03/www/8.02ch30we.pdf`. Geraadpleegd op 17 mei 2026.

[3] Wikipedia contributors, "Cross product — Wikipedia, the free encyclopedia." `https://en.wikipedia.org/wiki/Cross_product`, 2026. Geraadpleegd op 17 mei 2026.